OPERATING SYSTEMS

ASSIGNMENT 2 - CO2

Process Management, Scheduling, Synchronisation and Deadlock Handling

Name: B.Ashvin

Register Number: 192511235

Course: Operating System

Course Code: CSA0404

GitHub Repository:
https://github.com/ashvin280208/CSA-0404-Operating-systems.git

Total Marks: 50

Table of Contents

Table of Contents 2

0. Introduction and Environment 5

Repository structure 5

Q1 – Processes and Process States (6 Marks) 6

1.1 The problem 6

1.2 Process-state diagram 6

1.3 The process control block (PCB) 7

1.4 Program: creating processes with fork() and observing them 7

1.5 Where the time goes with 2,000 active processes 9

1.6 Judgement and design decision 12

1.7 Alternative concept: each session as a thread in one server process 13

1.8 Evidence summary for Q1 14

Q2 – CPU Scheduling (8 Marks) 14

2.1 The problem 14

2.2 Scheduling criteria and definitions 14

2.3 Simulation of all algorithms 14

2.4 The completed Round Robin program 21

2.5 Quantum study (1, 3, 5, 20) 23

2.6 Starvation and ageing 24

2.7 How a multilevel feedback queue treats short and long jobs 25

2.8 Design decision 25

2.9 Alternative concept and complexity: MLFQ instead of a single RR queue 25

2.10 Evidence summary for Q2 26

Q3 – Threads (6 Marks) 26

3.1 The problem 26

3.2 What threads share and what each has of its own 26

3.3 User-level and kernel-level threads, and the multithreading models 26

3.4 The program: processing 64 camera frames 27

3.5 The speed-up obtained and why it stops increasing 29
3.6 What happens to shared counters without protection 30
3.7 Alternative concept: several processes instead of threads 31
3.8 Evidence summary for Q3 32
Q4 – Inter-Process Communication (6 Marks) 33
4.1 The problem 33
4.2 Pipe implementation (completed code) 33
4.3 Shared-memory implementation 34
4.4 Differences between the models 36
4.5 Growth investigation: transfer of 1 MB, 10 MB and 100 MB 36
4.6 Alternative concept and complexity: shared memory with a semaphore instead of a pipe 39
4.7 Justified choice 40
4.8 Evidence summary for Q4 40
Q5 – Synchronisation: Critical Sections, Semaphores and Monitors (11 Marks) 40
5.1 The problem and the starting principle 40
5.2 From the principle to a design 41
5.3 Completed code: bounded buffer with semaphores and a mutex 46
5.4 Test of the design: 1 and 4 producers and consumers 47
5.5 Deployment decision: one lock, one lock per train, or one lock per coach 49
5.6 Alternative concept: atomic operations or spin locks instead of semaphores and mutexes 53
5.7 Evidence summary for Q5 54
Q6 – Deadlock Handling and the Banker's Algorithm (13 Marks) 54
A. Characterise Deadlock (2 Marks) 54
Complete the Code: the safety algorithm 55
Deadlock detection and recovery as the alternative concept 58
B. Compare the Strategies (3 Marks) 59
C. Apply the Banker's Algorithm (4 Marks) 60
D. Predict Before Implementing (1 Mark) 61
E. Measure (1 Mark) 61
F. Defend the Final Design (2 Marks) 63

Evidence summary for Q6 64

7. Conclusion of Assignment 2 **64**

Appendix A – Hand Workings for the Scheduling Algorithms (Q2) **65**

A.1 SJF (non-preemptive) 65

A.2 Priority scheduling (non-preemptive) 65

A.3 Round Robin with quantum 1 66

A.4 Why SRTF has the minimum average waiting time 66

Appendix B – Test Log of All Programs **67**

Appendix C – README.md Template **68**

Appendix D – Viva Preparation: Questions and Short Answers **68**

Appendix E – Extended Calculations **69**

0. Introduction and Environment

This report answers Assignment 2 of CSA04 – Operating Systems (Course Outcome CO2): process management, CPU scheduling, threads, inter-process communication, synchronisation and deadlock handling. Every question is at Bloom's level K5 (Evaluate), so each answer follows the cycle *model, design, implement, measure, compare, evaluate, recommend*. All C programs were compiled with `gcc -std=c11 -Wall -Wextra -pthread -O2` (gcc 13.3.0, Ubuntu 24.04, Linux 6.18) and executed; the printed outputs are the real outputs. Programs written in Python (scheduling simulation and wait-for graph) were run with Python 3.12.

> **Machine limitation.** The test machine is a virtual machine with **one CPU core**. This has a direct consequence for the thread and locking experiments: CPU-bound threads cannot run in parallel, so measured speed-ups are limited to about 1. This is reported honestly and is itself used as evidence (Q3: "why the speed-up stops increasing"). The commands should be re-run by the student on a multi-core machine and the larger speed-ups recorded in the repository for comparison.

Repository structure

```
CSA04-<RegisterNumber>-Assignment2/
  README.md   (name, register number, gcc 13.3.0, Ubuntu 24.04, compile command, run \
       steps, results)
  Q1/ q1_fork.c q1_ctx.c q1_mem.c      outputs/ ps_output.txt
  Q2/ q2_rr.c q2_sched.py              outputs/ rr_q1.txt rr_q3.txt rr_q5.txt rr_q20 \
      .txt
  Q3/ q3_cameras.c q3_race.c q3_proc.c outputs/
  Q4/ q4_pipe.c q4_noclose.c q4_shm.c q4_bench.c outputs/
  Q5/ q5_bb.c q5_bbmulti.c q5_lock.c q5_phil.c q5_rw.c q5_monitor.c outputs/
  Q6/ q6_banker.c q6_bench.c q6_detect.py outputs/
```

Commit history: one commit per question at least ("Q1: fork program and ps evidence", "Q2: round robin and scheduling study", and so on). The name, register number and link on the cover page are placeholders to be completed by the student.

Q1 – Processes and Process States (6 Marks)

1.1 The problem

An online examination server runs one process per student session, plus processes for auto-saving and proctoring. When 2,000 students log in together the server becomes slow and some sessions hang. The question is where the time goes and whether a process per student is a sound design.

1.2 Process-state diagram

Figure 1.1 Process-state diagram

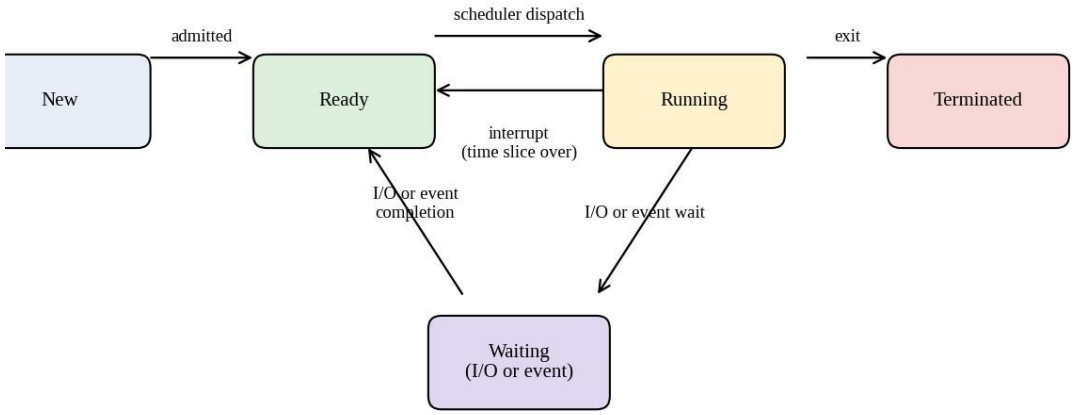


Figure 1.1 Process-state diagram

A process is a program in execution: its code, its current activity (program counter and registers), its stack, data section and heap. During its life it moves among five states.

- **New**: the process is being created (the PCB is allocated, memory is set up).
- **Ready**: the process is in memory and could run, but no CPU is free. It waits in the ready queue.
- **Running**: instructions are executing on a CPU.
- **Waiting (blocked)**: the process waits for an event such as the completion of a disk read, a network packet or a lock.
- **Terminated**: the process has finished; the kernel releases resources and keeps the exit status until the parent collects it.

The transitions are: *admitted* (new to ready); *scheduler dispatch* (ready to running); *interrupt* (running to ready, when the time slice expires or a higher-priority process arrives); *I/O or event wait* (running to waiting); *I/O or event completion* (waiting to ready); *exit* (running to terminated). Note that a waiting process never goes directly to running,

because the scheduler must choose it.

For the examination server this diagram explains "hanging" sessions. A session blocked on a slow disk write (auto-save) is in the waiting state; when 2,000 sessions complete I/O at nearly the same time they all become ready together and queue for a few CPUs, so some of them wait a long time in the ready queue. From the student's side that is a hang.

1.3 The process control block (PCB)

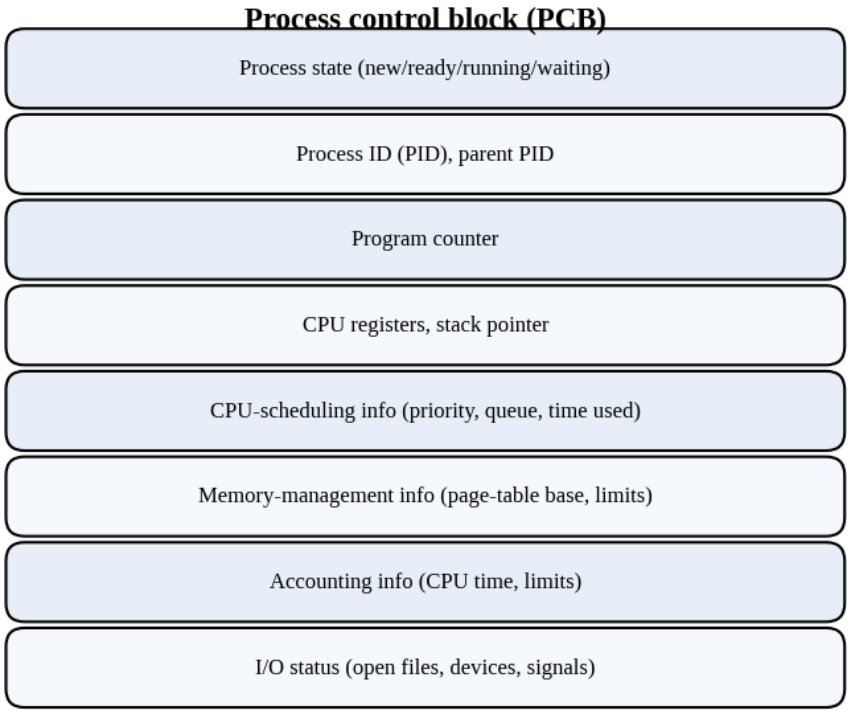


Figure 1.2 Contents of a process control block

The PCB (in Linux the `task_struct`) is the data structure through which the kernel represents a process. It contains the process state, the process number (PID and parent PID), the program counter and CPU registers saved at the last context switch, CPU-scheduling information (priority, scheduling class, queue pointers, time used), memory-management information (the page-table base, memory limits), accounting information (CPU time used, limits) and I/O status information (the table of open files, devices, pending signals). A context switch saves the registers into the PCB of the old process and loads them from the PCB of the new one.

1.4 Program: creating processes with fork() and observing them

The program below creates N child processes (each stands for a student session and simply sleeps), and then displays them with `ps` and with the `/proc` file system, which exposes the

kernel's PCB data.

Listing: q1_fork.c

```
#define _DEFAULT_SOURCE
/* Q1: create N child processes (one per "student session") and observe them with ps \
       and /proc. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main(int argc, char **argv)
{
    int n = argc > 1 ? atoi(argv[1]) : 5;
    pid_t kids[64];
    if (n > 64) n = 64;
    for (int i = 0; i < n; i++) {
        pid_t p = fork();
        if (p == 0) { sleep(3); _exit(i); }     /* the child "serves a session": it  \
      waits */
        kids[i] = p;
    }
    printf("parent %d created %d children; first child is %d\n", getpid(), n, kids[0 \
      ]);
    fflush(stdout);
    usleep(200000);
    char cmd[200];
    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,vsz,rss,nlwp,comm --ppid %d | hea \
      d -8", getpid());
    if (system(cmd)) {}
    printf("\n/proc/%d/status (excerpt = the PCB as seen by Linux):\n", kids[0]);
    fflush(stdout);
    snprintf(cmd, sizeof cmd, "grep -E '^(Name|State|Pid|PPid|Uid|VmSize|VmRSS|Threa \
      ds|voluntary_ctxt_switches|nonvoluntary_ctxt_switches)'  /proc/%d/status",  kids  \
      [0]);
    if (system(cmd)) {}
    printf("\n/proc/%d/stat fields: ", kids[0]); fflush(stdout);
    snprintf(cmd, sizeof cmd, "cut -d' ' -f1-8,14-15,19-20,23 /proc/%d/stat", kids[0 \
      ]);
    if (system(cmd)) {}
    int st, total = 0;
    for (int i = 0; i < n; i++) { waitpid(kids[i], &st, 0); total += WEXITSTATUS(st) \
      ; }
    printf("\nall children terminated; sum of exit codes = %d\n", total);
    return 0;
}
```

```
$ ./q1_fork 5

parent 117 created 5 children; first child is 118
  PID  PPID STAT    VSZ   RSS NLWP COMMAND
  118   117 S      2556   264    1 q1_fork
  119   117 S      2556   264    1 q1_fork
  120   117 S      2556   264    1 q1_fork
  121   117 S      2556   264    1 q1_fork
  122   117 S      2556   264    1 q1_fork
  123   117 S      2808  1920    1 sh

/proc/118/status (excerpt = the PCB as seen by Linux):
Name:    q1_fork
State:    S (sleeping)
Pid:    118
PPid:    117
Uid:    0    0    0    0
VmSize:        2556 kB
VmRSS:         264 kB
Threads:    1
voluntary_ctxt_switches:    1
nonvoluntary_ctxt_switches:    0

/proc/118/stat fields: 118 (q1_fork) S 117 111 0 0 -1 0 0 0 1 2617344

all children terminated; sum of exit codes = 10
```

Reading the evidence: `ps` shows five children with the same parent PID (the program), state `S` (sleeping, i.e. waiting), each using 2,556 KB of virtual memory (VSZ) and only 268 KB of resident memory (RSS): the rest of the address space is shared copy-on-write with the parent or not yet touched. `/proc/<pid>/status` shows the PCB fields the kernel keeps: state, PID, parent PID, user IDs, memory size, number of threads, and the counters of voluntary context switches (the process gave up the CPU by waiting) and non-voluntary context switches (the kernel took the CPU away). The `/proc/<pid>/stat` line shows the same information in compact form (PID, command, state S, parent, process group).

1.5 Where the time goes with 2,000 active processes

Three costs dominate: the cost of context switching, the memory used by each process, and the waiting time for I/O.

Context-switch cost

A context switch saves one process's registers, selects the next process, switches page tables (which flushes or partly invalidates the TLB) and restores registers. There is also an indirect cost: the CPU caches and TLB contain the old process's data, so the new process runs slowly until its working set is reloaded. The program `q1_ctx.c` measures the cost with two processes that pass one byte back and forth through two pipes; each round trip

needs two context switches.

Listing: q1_ctx.c

```
/* Q1/Q3: cost of a context switch between two processes and between two threads (pi \
      ng-pong through pipes). */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
#define ROUNDS 100000
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static int a[2], b[2];
static void *echo(void *x) { (void)x; char c; for (int i = 0; i < ROUNDS; i++) { if  \
      (read(a[0], &c, 1) != 1) break; if (write(b[1], &c, 1) != 1) break; } return N \
      ULL; }
int main(void)
{
    char c = 'x';
    if (pipe(a) || pipe(b)) return 1;
    pid_t p = fork();
    if (p == 0) { echo(NULL); _exit(0); }
    double t0 = now();
    for (int i = 0; i < ROUNDS; i++) { if (write(a[1], &c, 1) != 1 || read(b[0], &c, \
       1) != 1) break; }
    double t1 = now(); waitpid(p, NULL, 0);
    printf("processes: %d round trips in %.3f s -> %.2f us per switch\n", ROUNDS, t1 \
       - t0, (t1 - t0) * 1e6 / (2.0 * ROUNDS));
    pipe(a); pipe(b);
    pthread_t th; pthread_create(&th, NULL, echo, NULL);
    t0 = now();
    for (int i = 0; i < ROUNDS; i++) { if (write(a[1], &c, 1) != 1 || read(b[0], &c, \
       1) != 1) break; }
    t1 = now(); pthread_join(th, NULL);
    printf("threads  : %d round trips in %.3f s -> %.2f us per switch\n", ROUNDS, t1 \
       - t0, (t1 - t0) * 1e6 / (2.0 * ROUNDS));
    return 0;
}
```

$ ./q1_ctx

```
processes: 100000 round trips in 0.290 s -> 1.45 us per switch
threads  : 100000 round trips in 0.246 s -> 1.23 us per switch
```

On this machine a switch between two processes costs about 1.4 µs, and a switch between two threads of one process about 1.2 µs; the difference (about 15 %) is the cost of switching address spaces. The direct switch cost is small. However, if 2,000 runnable processes share 4 cores with a time slice of 4 ms, each process waits (2000/4) × 4 ms = 2 seconds between

turns, which the student perceives as a hang. A switch every 4 ms costs only 1.4 µs / 4 ms = 0.035 % direct overhead; the real loss is the cache pollution (each process finds the cache filled by 499 others), which can cost 10 to 30 % of throughput.

Memory per process

The program `q1_mem.c` measures the change in `MemAvailable` after creating 300 idle sessions as processes or as threads.

Listing: q1_mem.c

```
#define _DEFAULT_SOURCE
/* Q1: memory cost per idle session when it is a process and when it is a thread (Me \
      mAvailable difference). */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <pthread.h>
#include <sys/wait.h>
static long avail_kb(void) { FILE *f = fopen("/proc/meminfo", "r"); char l[128]; lon \
      g v = 0; while (fgets(l, sizeof l, f)) if (!strncmp(l, "MemAvailable:", 13)) { \
       v = atol(l + 13); break; } fclose(f); return v; }
static void *idle(void *x) { (void)x; char buf[2048]; memset(buf, 1, sizeof buf); sl \
      eep(5); return buf[0] ? NULL : NULL; }
int main(int argc, char **argv)
{
    int n = 300, proc = argc > 1 && argv[1][0] == 'p';
    long before = avail_kb();
    pid_t kids[400]; pthread_t th[400];
    for (int i = 0; i < n; i++) {
        if (proc) { pid_t p = fork(); if (p == 0) { char buf[2048]; memset(buf, 1, s \
      izeof buf); sleep(5); _exit(buf[0] ? 0 : 1); } kids[i] = p; }
        else { pthread_attr_t at; pthread_attr_init(&at); pthread_attr_setstacksize( \
      &at, 256 * 1024); pthread_create(&th[i], &at, idle, NULL); }
    }
    usleep(1500000);
    long after = avail_kb();
    printf("%s: %d idle sessions use %ld KB -> %.1f KB each\n", proc ? "processes" : \
       "threads  ", n, before - after, (double)(before - after) / n);
    fflush(stdout);
    for (int i = 0; i < n; i++) { if (proc) { kill(kids[i], SIGKILL); waitpid(kids[i \
      ], NULL, 0); } else pthread_detach(th[i]); }
    return 0;
}
```

```
$ ./q1_mem p
processes: 300 idle sessions use 34632 KB -> 115.4 KB each
$ ./q1_mem t
threads  : 300 idle sessions use 672 KB -> 2.2 KB each
```

The measurement has a resolution of a few kilobytes and is noisy (repeated runs gave 38 to 108 KB per process and 19 to 32 KB per thread), but both orders of magnitude are clear. A process costs a kernel structure and kernel stack (about 10 KB to 16 KB), page tables (at least 12 KB for a minimal program, more for large ones) and its private pages; a real session process of a web application with an interpreter, libraries and session data costs 5 to 50 MB. For 2,000 sessions at 20 MB that is 40 GB of memory for session data alone, compared with a few hundred MB if the sessions were light threads or event-driven tasks inside a few workers. When physical memory is exhausted, the OS starts swapping, the system slows by orders of magnitude, and sessions hang.

Waiting for I/O

Auto-save writes and proctoring video upload are I/O-bound. A process that waits for a disk write (about 100 µs on SSD, 10 ms on a hard disk) is in the waiting state and uses no CPU, so I/O waiting by itself does not need many processes; but 2,000 sessions that all save every 30 seconds create a burst of 2,000 writes at the same moment, and the disk queue grows (queueing delay = number waiting × service time = 2,000 × 100 µs = 200 ms on SSD, 20 s on a hard disk). The design must therefore smooth the bursts (batching, randomised save times, a write-behind cache).

1.6 Judgement and design decision

A separate process per student is **not a sound design** for 2,000 simultaneous students, because (a) each process carries a large memory overhead, (b) the scheduler must manage thousands of runnable tasks and the ready-queue delay grows linearly with their number, (c) process creation is slow (about 60 µs minimum for fork, up to milliseconds for a large server image; see Assignment 1 Task 10) and a login storm produces thousands of forks at the same time. The advantage of processes, strong isolation, is real, but it can be obtained more cheaply.

Option	Memory per session	Switch cost	Isolation	Scalability	Verdict
One process per student	High (MBs)	Highest (address-space switch)	Best: a crash affects one session	Poor above a few hundred	Rejected

Option	Memory per session	Switch cost	Isolation	Scalability	Verdict
One thread per student	Low (stack, 8 KB-1 MB, resident a few KB)	Lower	Weak: one fault can crash the whole server	Good to a few thousand; stack memory and scheduling still grow with N	Acceptable
Fixed pool of worker processes (for example 16 workers with threads or an event loop inside each)	Very low per session	Low, only W workers to schedule	Good between workers; a crash loses only the sessions of one worker	Excellent; bounded resources, queueing instead of overload	**Chosen**

Decision: use a fixed pool of worker processes (for example one per core or two per core), each handling many sessions with a small thread pool or an asynchronous event loop, with a separate small set of processes for auto-save and for proctoring. The pool size is chosen from the measured number of cores and memory. Excess requests wait in a bounded queue (the load balancer returns "busy" or queues them) rather than creating unlimited processes. This keeps the ready queue short, bounds memory, still isolates failures between workers, and restarts a crashed worker without affecting the others. Proctoring video is CPU-heavy and is separated into its own process pool so that it cannot starve answer saving.

1.7 Alternative concept: each session as a thread in one server process

What the solution would be. One server process creates a thread per session (`pthread_create`). All threads share the address space, so session data are ordinary memory and no IPC is needed.

Context-switch time. Thread switches within one process keep the same page tables, so no TLB flush is needed. The measurement above gives 1.23 µs against 1.45 µs for processes on this machine, i.e. about 15 % less; on hardware with expensive TLB flushes the gap is larger. In big-O terms a switch is O(1) in both cases, with a smaller constant for threads.

Memory per session. A thread needs a stack (virtual 8 MB by default, but only the touched pages, a few KB, are resident) and a small thread structure: measured 19 to 32 KB; a process needs a full address space description: measured 38 to 108 KB at minimum, and 5 to 50 MB for a real server image. For N sessions the memory is O(N × stack) for threads against O(N × address space) for processes; for N = 2,000 this is roughly 40 to 60 MB against 80 MB to 40 GB.

Effect of one session crashing. In the thread design a crash (for example a segmentation fault in one session's handler) kills the *whole process* and with it all 2,000 sessions, because the threads share one address space; a memory-corrupting bug in one session can silently corrupt another student's answers. In the process design only the crashing session is lost. This is why the thread-per-session design is rejected for the examination server and the worker-pool compromise is selected: threads inside a worker are cheap, and the isolation between workers limits the damage to (1/W) of the sessions.

1.8 Evidence summary for Q1

State diagram (Figure 1.1), PCB (Figure 1.2), a working fork() program with `ps` and `/proc` evidence, measured context-switch and memory costs, and a defended decision (fixed worker pool).

Q2 – CPU Scheduling (8 Marks)

2.1 The problem

A video-encoding service receives short jobs (thumbnails, 2 s) and long jobs (full films, 20 min) on the same servers. Customers complain that short jobs wait behind long ones, while the company wants the processors fully used. This is the *convoy effect* under FCFS: many short jobs queue behind one long job.

2.2 Scheduling criteria and definitions

For each job: *arrival time* a, *burst time* b (CPU time needed), *finish time* f. Then *turnaround time* TAT = f - a; *waiting time* WT = TAT - b (time spent in the ready queue); *response time* RT = (time of first execution) - a. The system is judged by CPU utilisation, throughput, and the *averages* of TAT, WT and RT. The same four jobs as in the code of the assignment are used: P1 (arrival 0, burst 8), P2 (1, 4), P3 (2, 9), P4 (3, 5). For the priority algorithm, priorities are assigned P1 = 2, P2 = 4, P3 = 1, P4 = 3 (1 is the highest).

2.3 Simulation of all algorithms

The Python program `q2_sched.py` simulates the five algorithms one time unit at a time and prints Gantt chart, finish time, TAT, WT and RT.

Listing: q2_sched.py

```
"""Q2: Gantt chart, waiting, turnaround and response times for FCFS, SJF, SRTF, Prio \
      rity and Round Robin,
plus a multilevel-feedback-queue simulation and an ageing demonstration."""
import collections

JOBS = [("P1", 0, 8, 2), ("P2", 1, 4, 4), ("P3", 2, 9, 1), ("P4", 3, 5, 3)]    # name \
```

```
      , arrival, burst, priority (1 = highest)

def compress(tl):
    segs = []
    for t, n in enumerate(tl):
        if n is None: continue
        if segs and segs[-1][0] == n and segs[-1][2] == t: segs[-1][2] = t + 1
        else: segs.append([n, t, t + 1])
    return segs

def metrics(jobs, tl):
    res = {}
    for n, a, b, p in jobs:
        fin = max(t for t, x in enumerate(tl) if x == n) + 1
        first = min(t for t, x in enumerate(tl) if x == n)
        tat = fin - a
        res[n] = (fin, tat, tat - b, first - a)
    return res

def sim(jobs, key, preemptive):
    rem = {n: b for n, a, b, p in jobs}; arr = {n: a for n, a, b, p in jobs}; pr = { \
      n: p for n, a, b, p in jobs}
    order = {n: i for i, (n, a, b, p) in enumerate(jobs)}
    tl = []; t = 0; cur = None
    while any(rem.values()):
        ready = [n for n in rem if rem[n] > 0 and arr[n] <= t]
        if not ready: tl.append(None); t += 1; continue
        if preemptive or cur is None or rem[cur] == 0:
            cur = min(ready, key=lambda n: (key(n, rem, arr, pr), arr[n], order[n]))
        tl.append(cur); rem[cur] -= 1; t += 1
    return tl

def rr(jobs, q):
    rem = {n: b for n, a, b, p in jobs}; tl = []; t = 0; queue = collections.deque() \
      ; added = set()
    def admit():
        for n, a, b, p in jobs:
            if n not in added and a <= t: queue.append(n); added.add(n)
    admit()
    while any(rem.values()):
        if not queue: tl.append(None); t += 1; admit(); continue
        n = queue.popleft(); run = min(q, rem[n])
        for _ in range(run): tl.append(n); t += 1
        rem[n] -= run; admit()
        if rem[n] > 0: queue.append(n)
    return tl

def report(title, jobs, tl):
    m = metrics(jobs, tl)
    print(title)
    print("  Gantt: " + " ".join("%s(%d-%d)" % tuple(s) for s in compress(tl)))
```

```
    print("  %-4s %6s %5s %5s %5s" % ("Job", "Finish", "TAT", "WT", "RT"))
    for n in sorted(m): print("  %-4s %6d %5d %5d %5d" % ((n,) + m[n]))
    k = len(m)
    print("  Averages: TAT = %.2f  WT = %.2f  RT = %.2f   context switches = %d\n" % \
       (
        sum(v[1] for v in m.values()) / k, sum(v[2] for v in m.values()) / k, sum(v[ \
      3] for v in m.values()) / k, len(compress(tl)) - 1))
    return m

print("Jobs (name, arrival, burst, priority):", JOBS, "\n")
report("FCFS", JOBS, sim(JOBS, lambda n, r, a, p: a[n], False))
report("SJF (non-preemptive)", JOBS, sim(JOBS, lambda n, r, a, p: r[n], False))
report("SRTF (preemptive SJF)", JOBS, sim(JOBS, lambda n, r, a, p: r[n], True))
report("Priority (non-preemptive, 1 = highest)", JOBS, sim(JOBS, lambda n, r, a, p:  \
     p[n], False))
for q in (1, 3, 5, 20): report("Round Robin, quantum %d" % q, JOBS, rr(JOBS, q))

# ---- Quantum study on a mixed workload: 1 long film + 6 short thumbnails
print("=" * 70)
MIX = [("Film", 0, 60, 3)] + [("T%d" % i, 2 * i, 2, 1) for i in range(1, 7)]
print("Mixed workload (1 long job of 60 units and 6 short jobs of 2 units):")
for name, tl in [("FCFS", sim(MIX, lambda n, r, a, p: a[n], False)), ("SJF", sim(MIX \
     , lambda n, r, a, p: r[n], False)),
                 ("SRTF", sim(MIX, lambda n, r, a, p: r[n], True))] + [("RR q=%d" %  \
     q, rr(MIX, q)) for q in (1, 3, 5, 20)]:
    m = metrics(MIX, tl); short = [m[n][2] for n in m if n != "Film"]
    print("  %-8s avg WT short = %6.2f   WT long = %3d   avg WT all = %6.2f   switch \
     es = %d" % (
       name, sum(short) / len(short), m["Film"][2], sum(v[2] for v in m.values()) / \
      len(m), len(compress(tl)) - 1))

# ---- Multilevel feedback queue
def mlfq(jobs, quanta=(2, 4, 8), boost=None):
    rem = {n: b for n, a, b, p in jobs}; arr = {n: a for n, a, b, p in jobs}
    level = {n: 0 for n in rem}; qs = [collections.deque() for _ in quanta]; added = \
       set(); tl = []; t = 0
    used = 0; cur = None
    def admit():
        for n, a, b, p in jobs:
            if n not in added and a <= t: qs[0].append(n); added.add(n)
    while any(rem.values()):
        admit()
        if boost and t > 0 and t % boost == 0:
            for lv in range(1, len(qs)):
                while qs[lv]: n = qs[lv].popleft(); level[n] = 0; qs[0].append(n)
            if cur is not None and level[cur] > 0: level[cur] = 0
        top = next((i for i, q in enumerate(qs) if q), None)
        if cur is not None and top is not None and top < level[cur]:     # preempted \
       by a higher queue
            qs[level[cur]].appendleft(cur); cur = None; used = 0
        if cur is None:
```

```
            if top is None: tl.append(None); t += 1; continue
            cur = qs[top].popleft(); used = 0
        tl.append(cur); rem[cur] -= 1; used += 1; t += 1
        if rem[cur] == 0: cur = None
        elif used >= quanta[level[cur]]:
            level[cur] = min(level[cur] + 1, len(qs) - 1); qs[level[cur]].append(cur \
      ); cur = None
    return tl
m = metrics(MIX, mlfq(MIX)); short = [m[n][2] for n in m if n != "Film"]
print("  %-8s avg WT short = %6.2f   WT long = %3d   avg WT all = %6.2f   switches = \
       %d" % ("MLFQ", sum(short) / len(short), m["Film"][2], sum(v[2] for v in m.val \
      ues()) / len(m), len(compress(mlfq(MIX))) - 1))

# ---- Starvation and ageing
print("=" * 70)
print("Starvation demo: one long job (10 units, arrives at 1) and a stream of short  \
      jobs (2 units) arriving every 2 units from t=0 until t=60.")
STR = [("Long", 1, 10, 5)] + [("S%02d" % i, 2 * (i - 1), 2, 1) for i in range(1, 32) \
      ]
for label, key in [("SJF without ageing", lambda n, r, a, p: r[n]), ("Priority witho \
      ut ageing", lambda n, r, a, p: p[n])]:
    tl = sim(STR, key, False); m = metrics(STR, tl); print("  %-26s Long job waits % \
      3d units (finishes at %d)" % (label, m["Long"][2], m["Long"][0]))
def sim_age(jobs, rate):
    rem = {n: b for n, a, b, p in jobs}; arr = {n: a for n, a, b, p in jobs}; tl = [ \
      ]; t = 0; cur = None
    while any(rem.values()):
        ready = [n for n in rem if rem[n] > 0 and arr[n] <= t]
        if not ready: tl.append(None); t += 1; continue
        if cur is None or rem[cur] == 0: cur = min(ready, key=lambda n: (rem[n] - ra \
      te * (t - arr[n]), arr[n]))
        tl.append(cur); rem[cur] -= 1; t += 1
    return tl
for rate in (0.0, 0.25, 0.5):
    m = metrics(STR, sim_age(STR, rate)); print("  SJF with ageing rate %.2f  Long  \
      job waits %3d units (finishes at %d)" % (rate, m["Long"][2], m["Long"][0]))
```

$ python3 q2_sched.py

```
Jobs (name, arrival, burst, priority): [('P1', 0, 8, 2), ('P2', 1, 4, 4), ('P3', 2,  \
      9, 1), ('P4', 3, 5, 3)]

FCFS
  Gantt: P1(0-8) P2(8-12) P3(12-21) P4(21-26)
  Job  Finish   TAT    WT    RT
  P1        8     8     0     0
  P2       12    11     7     7
  P3       21    19    10    10
  P4       26    23    18    18
  Averages: TAT = 15.25  WT = 8.75  RT = 8.75   context switches = 3

SJF (non-preemptive)
```

```
  Gantt: P1(0-8) P2(8-12) P4(12-17) P3(17-26)
  Job   Finish    TAT    WT    RT
  P1         8     8     0     0
  P2        12    11     7     7
  P3        26    24    15    15
  P4        17    14     9     9
  Averages: TAT = 14.25  WT = 7.75  RT = 7.75   context switches = 3

SRTF (preemptive SJF)
  Gantt: P1(0-1) P2(1-5) P4(5-10) P1(10-17) P3(17-26)
  Job   Finish    TAT    WT    RT
  P1        17    17     9     0
  P2         5     4     0     0
  P3        26    24    15    15
  P4        10     7     2     2
  Averages: TAT = 13.00  WT = 6.50  RT = 4.25   context switches = 4

Priority (non-preemptive, 1 = highest)
  Gantt: P1(0-8) P3(8-17) P4(17-22) P2(22-26)
  Job   Finish    TAT    WT    RT
  P1         8     8     0     0
  P2        26    25    21    21
  P3        17    15     6     6
  P4        22    19    14    14
  Averages: TAT = 16.75  WT = 10.25  RT = 10.25   context switches = 3

Round Robin, quantum 1
   Gantt: P1(0-1) P2(1-2) P1(2-3) P3(3-4) P2(4-5) P4(5-6) P1(6-7) P3(7-8) P2(8-9) P4( \
       9-10) P1(10-11) P3(11-12) P2(12-13) P4(13-14) P1(14-15) P3(15-16) P4(16-17) P1 \
      (17-18) P3(18-19) P4(19-20) P1(20-21) P3(21-22) P1(22-23) P3(23-26)
  Job   Finish    TAT    WT    RT
  P1        23    23    15     0
  P2        13    12     8     0
  P3        26    24    15     1
  P4        20    17    12     2
  Averages: TAT = 19.00  WT = 12.50  RT = 0.75   context switches = 23

Round Robin, quantum 3
   Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1 \
       (21-23) P3(23-26)
  Job   Finish    TAT    WT    RT
  P1        23    23    15     0
  P2        16    15    11     2
  P3        26    24    15     4
  P4        21    18    13     6
  Averages: TAT = 20.00  WT = 13.50  RT = 3.00   context switches = 9

Round Robin, quantum 5
  Gantt: P1(0-5) P2(5-9) P3(9-14) P4(14-19) P1(19-22) P3(22-26)
  Job   Finish    TAT    WT    RT
  P1        22    22    14     0
```

```
  P2         9     8     4     4
  P3        26    24    15     7
  P4        19    16    11    11
  Averages: TAT = 17.50  WT = 11.00  RT = 5.50   context switches = 5

Round Robin, quantum 20
  Gantt: P1(0-8) P2(8-12) P3(12-21) P4(21-26)
  Job  Finish   TAT    WT    RT
  P1         8     8     0     0
  P2        12    11     7     7
  P3        21    19    10    10
  P4        26    23    18    18
  Averages: TAT = 15.25  WT = 8.75  RT = 8.75   context switches = 3

======================================================================
Mixed workload (1 long job of 60 units and 6 short jobs of 2 units):
  FCFS     avg WT short =  58.00   WT long =   0   avg WT all =  49.71   switches =  \
      6
  SJF      avg WT short =  58.00   WT long =   0   avg WT all =  49.71   switches =  \
      6
  SRTF     avg WT short =   0.00   WT long =  12   avg WT all =   1.71   switches =  \
      7
  RR q=1   avg WT short =   3.83   WT long =  12   avg WT all =   5.00   switches =  \
      18
  RR q=3   avg WT short =   4.50   WT long =  12   avg WT all =   5.57   switches =  \
      9
  RR q=5   avg WT short =   6.33   WT long =  12   avg WT all =   7.14   switches =  \
      8
  RR q=20  avg WT short =  18.00   WT long =  12   avg WT all =  17.14   switches =  \
      7
  MLFQ     avg WT short =   0.00   WT long =  12   avg WT all =   1.71   switches =  \
      7
======================================================================
Starvation demo: one long job (10 units, arrives at 1) and a stream of short jobs (2 \
       units) arriving every 2 units from t=0 until t=60.
  SJF without ageing         Long job waits  61 units (finishes at 72)
  Priority without ageing    Long job waits  61 units (finishes at 72)
  SJF with ageing rate 0.00   Long job waits  61 units (finishes at 72)
  SJF with ageing rate 0.25   Long job waits  33 units (finishes at 44)
  SJF with ageing rate 0.50   Long job waits  17 units (finishes at 28)
```

Figure 2.1 Gantt charts for the four jobs

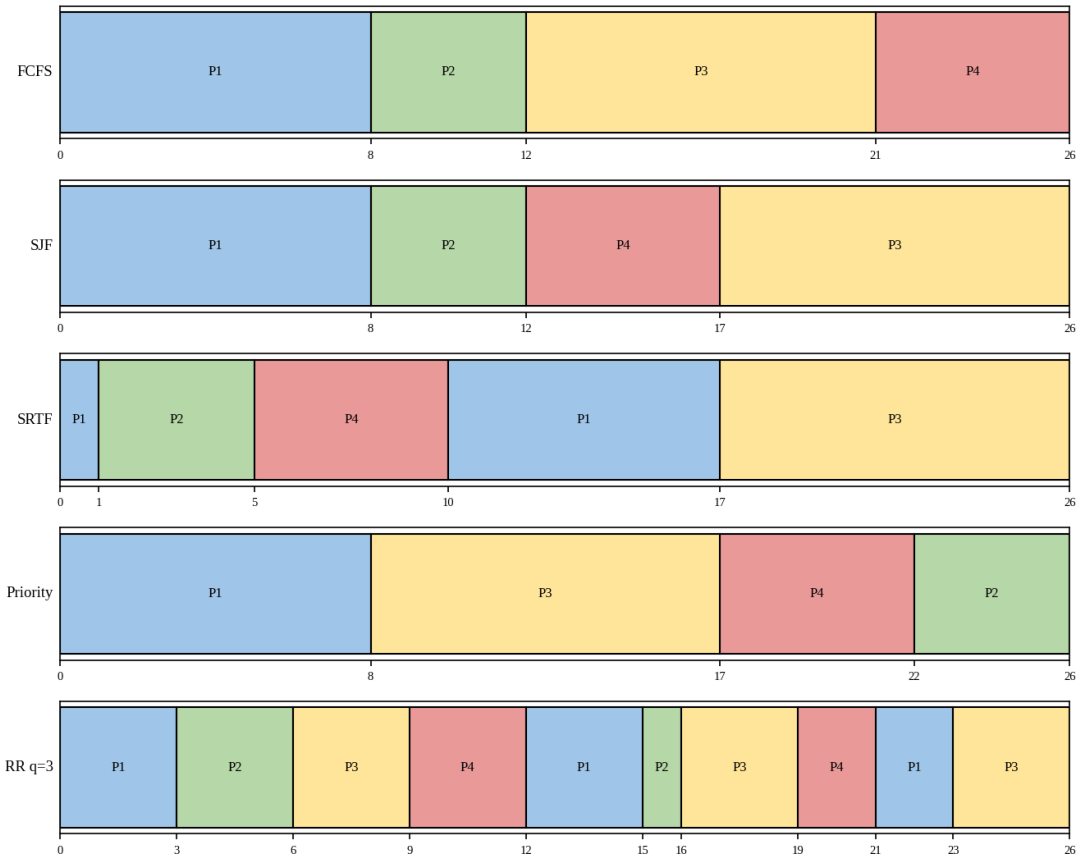


Figure 2.1 Gantt charts

Hand verification of two algorithms

FCFS. Order of arrival P1, P2, P3, P4. P1 runs 0–8, P2 8–12, P3 12–21, P4 21–26. Waiting times: P1 = 0, P2 = 8 - 1 = 7, P3 = 12 - 2 = 10, P4 = 21 - 3 = 18, average = 35/4 = 8.75. Turnaround: 8, 11, 19, 23, average 15.25. Matches the program.

SRTF. At t = 0 only P1 is present and runs. At t = 1 P2 arrives with burst 4 < remaining 7 of P1, so P2 preempts P1 (1–5). At t = 3 P4 (5) arrives but P2 has 2 left, so P2 continues. At t = 5 the choice is P4 (5), P1 (7), P3 (9): P4 runs 5–10. Then P1 (10–17), then P3 (17–26). WT: P1 = 17 - 0 - 8 = 9, P2 = 5 - 1 - 4 = 0, P3 = 26 - 2 - 9 = 15, P4 = 10 - 3 - 5 = 2; average 26/4 = 6.5. Matches. SRTF gives the minimum average waiting time of all algorithms (it is optimal), but needs the burst times in advance.

Comparison table

Algorithm	Average TAT	Average WT	Average RT	Context switches	Remarks
FCFS	15.25	8.75	8.75	3	Simple, convoy effect; short job P2 waits 7

Algorithm	Average TAT	Average WT	Average RT	Context switches	Remarks
SJF (non-preemptive)	14.25	7.75	7.75	3	Needs burst estimates; can starve long jobs
SRTF	13.00	6.50	4.25	4	Best average WT; most preemptions
Priority (non-preemptive)	16.75	10.25	10.25	3	Low-priority P2 waits 21; starvation risk
Round Robin q = 1	19.00	12.50	0.75	23	Best response; many switches
Round Robin q = 3	20.00	13.50	3.00	9	The assignment's configuration
Round Robin q = 5	17.50	11.00	5.50	5	
Round Robin q = 20	15.25	8.75	8.75	3	Degenerates into FCFS

2.4 The completed Round Robin program

TODO 1 was completed as follows: run process p for min(quantum, left[p]); print its Gantt segment; advance the clock; reduce the remaining time; admit every process that has arrived by the new time and is not yet in the queue (arrivals are admitted *before* the preempted process is put back, which is the usual convention and gives the expected Gantt chart); then re-queue p if it still has work, otherwise record its finish time. The program has a `QUANTUM` macro (default 3) so that other quanta can be compiled without changing the code; a counter of context switches was added after the loop body for the quantum study.

Listing: q2_rr.c

```
#include <stdio.h>
#define N 4
#ifndef QUANTUM
#define QUANTUM 3
#endif
int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = QUANTUM;
    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0, switches = 0, last = -1;
```

```
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        /* TODO 1 (completed) */
        int run = left[p] < quantum ? left[p] : quantum;    /* min(quantum, left[p])  \
          */
        printf(" %s(%d-%d)", name[p], time, time + run);   /* print the Gantt segmen \
      t   */
        time += run;                                       /* advance the clock      \
          */
        left[p] -= run;                                    /* reduce remaining time  \
          */
        for (int i = 0; i < N; i++)                        /* admit new arrivals fir \
      st  */
            if (!added[i] && arrival[i] <= time) { queue[tail++] = i; added[i] = 1;  \
      }
        if (left[p] > 0) queue[tail++] = p;                /* back to the end of que \
      ue  */
        else { finish[p] = time; done++; }                 /* finished: record the t \
      ime */
        if (last >= 0 && last != p) switches++;            /* CPU changed process    \
          */
        last = p;
    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    printf("(context switches = %d)\n", switches);
    return 0;
}
```

$ gcc -std=c11 -Wall -Wextra -o q2_rr q2_rr.c && ./q2_rr

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(2 \
      1-23) P3(23-26)
Process TAT WT
P1        23  15
P2        15  11
P3        24  15
P4        18  13
Average TAT = 20.00, average WT = 13.50
(context switches = 9)
```

The output is exactly the expected output: Gantt chart `P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)`, TAT 23, 15, 24, 18, WT 15, 11, 15, 13, average TAT 20.00 and average WT 13.50.

2.5 Quantum study (1, 3, 5, 20)

```
$ gcc -std=c11 -DQUANTUM=1 -o q2_rr1 q2_rr.c && ./q2_rr1 | tail -3
```

```
P4        17  12
Average TAT = 19.00, average WT = 12.50
(context switches = 23)
```

```
$ gcc -std=c11 -DQUANTUM=5 -o q2_rr5 q2_rr.c && ./q2_rr5 | tail -3
```

```
P4        16  11
Average TAT = 17.50, average WT = 11.00
(context switches = 5)
```

```
$ gcc -std=c11 -DQUANTUM=20 -o q2_rr20 q2_rr.c && ./q2_rr20
```

```
Gantt: P1(0-8) P2(8-12) P3(12-21) P4(21-26)
Process TAT WT
P1         8   0
P2        11   7
P3        19  10
P4        23  18
Average TAT = 15.25, average WT = 8.75
(context switches = 3)
```

Figure 2.2 Effect of the time quantum on Round Robin

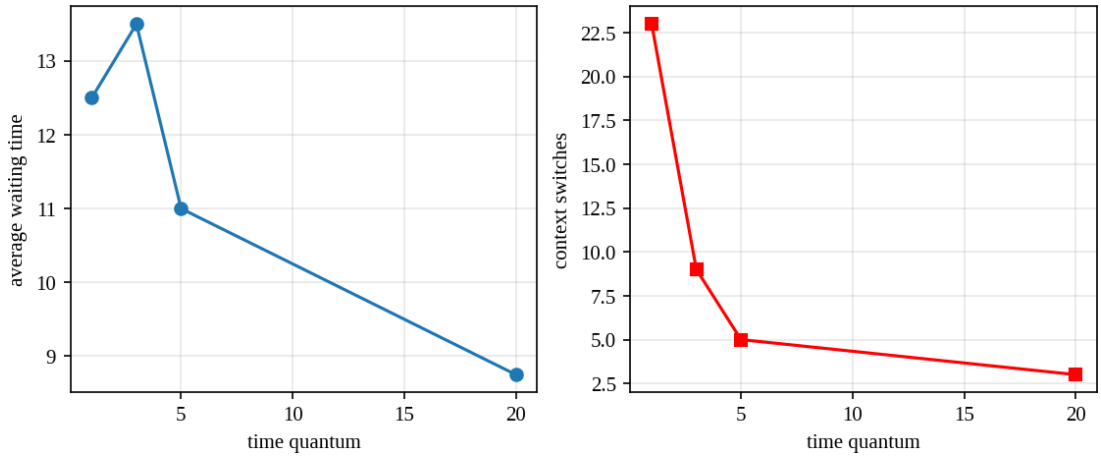


Figure 2.2 Average waiting time and context switches against the quantum

Observations. With quantum 1 the response time is the best (average 0.75) but there are 23 context switches for 26 time units of work; if a switch costs 1 µs against a quantum of 1 µs the overhead is 100 %. With quantum 20, which is larger than every burst, Round Robin becomes FCFS: 3 switches, average WT 8.75 (this is the best average WT for this particular set only because the short jobs happen to arrive after the shortest long job; for a

mix with a long first job FCFS is bad, as the next experiment shows). The average waiting time is not monotonic in q (12.50, 13.50, 11.00, 8.75) because of the arrival pattern, but the context-switch count falls steadily (23, 9, 5, 3). The rule of thumb is that the quantum should be larger than the context-switch time by a factor of at least 100 and large enough that about 80 % of CPU bursts finish within one quantum (Linux time slices are of the order of 1 to 10 ms).

Mixed workload: one film and six thumbnails

The same simulator was used for the video service: one long job of 60 units arrives at t = 0, and six jobs of 2 units arrive at t = 2, 4, ..., 12. The averages for the short jobs are in Table 2.2 (the output is shown in the program run above).

Policy	Average WT of short jobs	WT of the long job	Average WT all	Context switches
FCFS	58.00	0	49.71	6
SJF (non-preemptive)	58.00	0	49.71	6
SRTF	0.00	12	1.71	7
Round Robin q = 1	3.83	12	5.00	18
Round Robin q = 3	4.50	12	5.57	9
Round Robin q = 5	6.33	12	7.14	8
Round Robin q = 20	18.00	12	17.14	7
Multilevel feedback queue (2, 4, 8)	0.00	12	1.71	7

This quantifies the customers' complaint: under FCFS a thumbnail waits 58 time units on average (an average of 58 s for a 2 s job); with preemption (SRTF, RR with a small quantum, or MLFQ) it waits 0 to 6 time units, and the long job loses only 12 units (the total work of the short jobs).

2.6 Starvation and ageing

SJF and priority scheduling can **starve** a job: as long as shorter (or higher-priority) jobs keep arriving, the long (or low-priority) job is never chosen. The last part of the simulation creates a stream of 2-unit jobs arriving every 2 units from t = 0 to t = 60 and one 10-unit job arriving at t = 1. Without ageing, the long job waits **61 units** (it starts only after the stream ends at t = 62), under both SJF and priority scheduling. **Ageing** gradually improves the priority of a job that has waited: here the effective key is remaining time - rate × waiting time. With rate 0.25 the long job waits 33 units and with rate 0.5 only 17 units. Ageing therefore turns an unbounded wait into a bounded one, at the price of slightly delaying the

short jobs.

2.7 How a multilevel feedback queue treats short and long jobs

A multilevel feedback queue (MLFQ) has several ready queues, for example Q0 with quantum 2, Q1 with quantum 4 and Q2 with quantum 8 (or FCFS). A new job enters Q0. If it finishes within its quantum it leaves (short jobs finish quickly and wait very little). If it uses the whole quantum it is moved down to the next queue (it is assumed to be long). A lower queue runs only when all higher queues are empty, and a newly arrived job preempts a lower-queue job. Long jobs therefore sink to the lowest queue, where they get large quanta (fewer context switches) but low priority. To prevent starvation, jobs that wait too long in a low queue are promoted (ageing or a periodic boost of all jobs to Q0). In the mixed workload simulation MLFQ gave the short jobs average waiting time 0, equal to SRTF, *without knowing the burst times in advance*: it learns the length of a job from its behaviour.

2.8 Design decision

Recommendation: a multilevel feedback queue with ageing (short quantum for new jobs, doubling quanta for lower queues, periodic boost). It gives the thumbnails an average waiting time of about 0 compared with 58 under FCFS (the calculated averages above), keeps the long films running with 12 units lost on 60 (20 %), uses the processors fully because a lower queue runs whenever the higher ones are empty, and needs no burst predictions. SRTF/SJF is best on paper but needs burst times (the user can lie or estimate wrongly) and starves films; plain Round Robin with a small quantum is fair but costs many context switches for the long jobs and gives the thumbnails 3.8 to 18 units of waiting depending on the quantum. In practice the service should additionally run thumbnails on a separate dedicated pool of workers to guarantee their latency.

2.9 Alternative concept and complexity: MLFQ instead of a single RR queue

Average waiting time for short and long jobs. From the table: single RR (q = 3) gives short jobs 4.5 and the long job 12; MLFQ gives short jobs 0.0 and the long job 12. So MLFQ is better for the short jobs and no worse for the long job in this workload. In general, MLFQ approximates SJF without knowing the bursts.

Scheduling cost per decision. RR picks the head of one queue: O(1). MLFQ with k queues picks the first non-empty queue: O(k) for a fixed small k, that is O(1) per decision (a bitmap of non-empty queues makes it truly constant, as in the Linux O(1) scheduler of version 2.6). SJF must find the shortest job among n ready jobs: O(n) with a list or O(log n)

with a heap. For 2,000 ready jobs the SJF decision examines up to 2,000 jobs, against 3 queue heads for MLFQ.

Risk of starvation. MLFQ without a boost can starve jobs in the lowest queue if short jobs keep arriving; with a periodic boost (every S time units all jobs return to Q0) the maximum wait of any job is bounded. The price is that a job can game the system by yielding just before its quantum expires, which the boost and per-level time accounting reduce. Space: O(n) in both cases (the queues hold the n ready jobs), plus O(k) for the queue heads.

2.10 Evidence summary for Q2

Correct Gantt charts and averages for all five algorithms (Section 2.3), a working Round Robin program (Section 2.4), the quantum study (Section 2.5), the starvation/ageing demonstration (Section 2.6) and the justified recommendation (Section 2.8).

Q3 – Threads (6 Marks)

3.1 The problem

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind at peak hours. The solution is to process frames in parallel with POSIX threads.

3.2 What threads share and what each has of its own

A thread is a unit of CPU execution within a process. All threads of a process **share**: the code (text) section, the global and static data, the heap, the open files and sockets, the signal handlers, the current directory, the process ID and the user credentials. Each thread has its **own**: thread ID, program counter, register set, **stack** (local variables and return addresses), thread-local storage, signal mask and scheduling priority. In the kernel the thread is a separate schedulable entity (in Linux a `task_struct` created by `clone` with the flags that share the address space and the file table).

Consequences: communication between threads is simple (ordinary memory), creation and switching are cheap, but there is no protection between threads, so shared data must be synchronised, and a fault in one thread terminates the whole process.

3.3 User-level and kernel-level threads, and the multithreading models

User-level threads are managed by a library in user space; the kernel does not know about them. They are fast to create and switch (no system call), but if one thread makes a blocking system call, the whole process blocks, and they cannot run on several CPUs at once. *Kernel-level threads* are managed and scheduled by the OS: a blocking call blocks

only one thread and threads run in parallel on different CPUs, but creation and switching need system calls and cost more.

Model	Mapping	Advantages	Disadvantages	Example
Many-to-one	Many user threads to one kernel thread	Very cheap thread management; no kernel support needed	One blocking call blocks all; no parallelism	Early Java green threads, GNU Portable Threads
One-to-one	Each user thread to its own kernel thread	True parallelism; blocking affects one thread	Each thread costs a kernel thread, which limits the number	Linux (NPTL), Windows, macOS
Many-to-many	Many user threads multiplexed onto a smaller or equal number of kernel threads	Many threads and parallelism; no limit problem	Complex to implement; scheduler cooperation needed	Solaris (old), Go runtime goroutines, Windows UMS

POSIX threads on Linux use the one-to-one model, which is the model assumed in the program below.

3.4 The program: processing 64 camera frames

The simulated frame processing consists of an optional I/O wait (reading the frame from the camera, 5 ms) and a CPU-bound number-plate recognition loop (about 5 ms of CPU). The program runs the frames sequentially, with a pool of 4, 8 and 16 threads (the threads fetch the next frame number from a shared atomic counter), and with one thread per camera (64 threads). It runs each configuration for two workloads: CPU only, and I/O plus CPU.

Listing: q3_cameras.c

```
/* Q3: process 64 camera frames: sequentially, one thread per camera, and with pools \
       of 4, 8 and 16 threads.
   Each frame = (optional) I/O wait for the camera + CPU work for number-plate recog \
      nition (simulated). */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#include <unistd.h>
#include <stdatomic.h>
#define CAMERAS 64
static int io_ms;                       /* simulated camera/network wait per frame * \
      /
static long work_iters = 4000000;       /* simulated recognition work per frame    * \
      /
```

```
static atomic_long plates_found;        /* shared result counter (protected by atomi \
      cs) */
static atomic_int next_frame;
static volatile unsigned long sink;   /* prevents the compiler from deleting the wor \
      k loop */
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static void process_frame(int id)
{
    if (io_ms) { struct timespec t = {0, io_ms * 1000000L}; nanosleep(&t, NULL); }
    unsigned long h = 14695981039346656O3UL + id;
    for (long i = 0; i < work_iters; i++) h = (h ^ (unsigned long)i) * 1099511628211 \
      UL;   /* CPU-bound loop */
    sink = h;                                   /* use the result so that the loop i \
      s really executed */
    atomic_fetch_add(&plates_found, 1);         /* one more frame done: shared count \
      er, atomic update */
}
static void *one(void *a) { process_frame((int)(long)a); return NULL; }
static void *pool_worker(void *a) { (void)a; int i; while ((i = atomic_fetch_add(&ne \
      xt_frame, 1)) < CAMERAS) process_frame(i); return NULL; }
static double run_seq(void) { double t0 = now(); for (int i = 0; i < CAMERAS; i++) p \
      rocess_frame(i); return now() - t0; }
static double run_per_camera(void) { pthread_t t[CAMERAS]; double t0 = now(); for (l \
      ong i = 0; i < CAMERAS; i++) pthread_create(&t[i], NULL, one, (void *)i); for  \
      (int i = 0; i < CAMERAS; i++) pthread_join(t[i], NULL); return now() - t0; }
static double run_pool(int n) { pthread_t t[64]; atomic_store(&next_frame, 0); doubl \
      e t0 = now(); for (int i = 0; i < n; i++) pthread_create(&t[i], NULL, pool_wor \
      ker, NULL); for (int i = 0; i < n; i++) pthread_join(t[i], NULL); return now() \
       - t0; }
int main(void)
{
    int ios[] = {0, 5};
    for (int k = 0; k < 2; k++) {
        io_ms = ios[k];
        printf("\nWorkload: %s\n%-26s %9s %9s\n", io_ms ? "5 ms I/O wait + CPU work  \
      per frame" : "CPU work only", "method", "time (s)", "speed-up");
        double seq = run_seq();
        printf("%-26s %9.3f %9.2f\n", "sequential (1 thread)", seq, 1.0);
        double d;
        d = run_pool(4);  printf("%-26s %9.3f %9.2f\n", "pool of 4 threads", d, seq  \
      / d);
        d = run_pool(8);  printf("%-26s %9.3f %9.2f\n", "pool of 8 threads", d, seq  \
      / d);
        d = run_pool(16); printf("%-26s %9.3f %9.2f\n", "pool of 16 threads", d, seq \
       / d);
        d = run_per_camera(); printf("%-26s %9.3f %9.2f\n", "one thread per camera ( \
      64)", d, seq / d);
    }
    printf("\nframes processed in all runs: %ld (expected %d)\n", (long)atomic_load( \
      &plates_found), CAMERAS * 10);
```

```
    return 0;
}
```

```
$ ./q3_cameras
```

```
Workload: CPU work only
method                          time (s)   speed-up
sequential (1 thread)              0.349       1.00
pool of 4 threads                  0.341       1.02
pool of 8 threads                  0.324       1.08
pool of 16 threads                 0.328       1.06
one thread per camera (64)         0.332       1.05

Workload: 5 ms I/O wait + CPU work per frame
method                          time (s)   speed-up
sequential (1 thread)              0.663       1.00
pool of 4 threads                  0.332       2.00
pool of 8 threads                  0.334       1.98
pool of 16 threads                 0.345       1.92
one thread per camera (64)         0.331       2.00

frames processed in all runs: 640 (expected 640)
```

Figure 3.1 Measured speed-up on a one-core machine

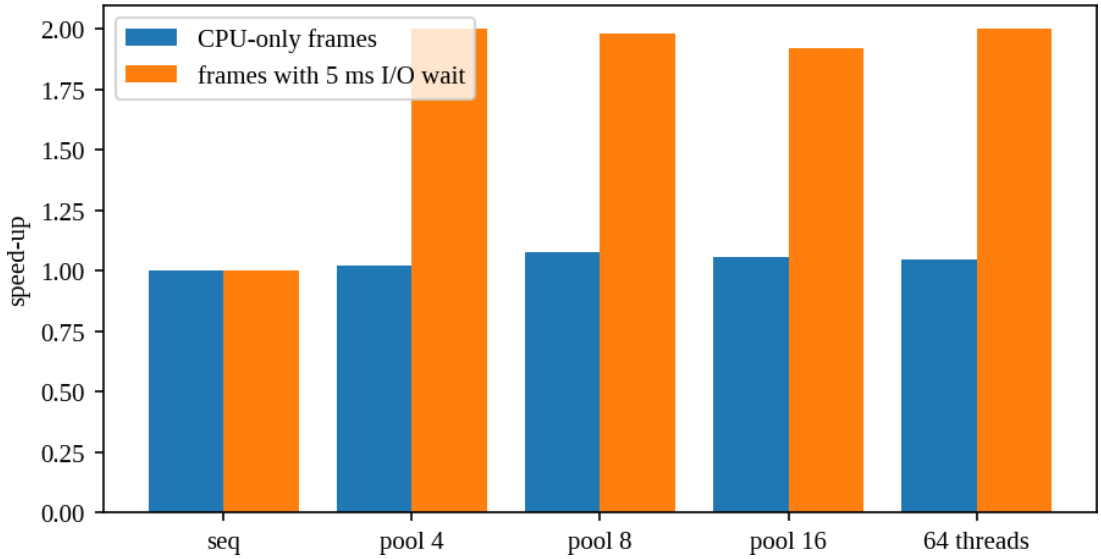


Figure 3.1 Measured speed-up for the thread pools

3.5 The speed-up obtained and why it stops increasing

CPU-only frames: speed-up 1.0 to 1.08. The machine has *one* core, so all threads share the same processor, and the total CPU work is unchanged; threads only add overhead. The speed-up is bounded by the number of cores p. On a machine with p cores, the best possible speed-up for pure CPU work is p (Amdahl: S(p) = 1/(s + (1 - s)/p), where s is the serial fraction, here the thread creation, the final result collection and the memory bandwidth limit).

Frames with I/O wait: speed-up 2.0. Here threads help even on one core, because while one thread waits for its camera (5 ms) another can compute. The ideal speed-up is limited by (I/O time + CPU time) / max(I/O time, CPU time) = (5 + 5)/5 = 2, which is exactly what is measured for 4, 8, 16 or 64 threads. Once the CPU is 100 % busy, more threads cannot help: the speed-up "stops increasing" as soon as the bottleneck resource (the CPU) is saturated.

Pool size. Pools of 4, 8 and 16 gave the same result (1.9 to 2.0), and one thread per camera (64 threads) gave no further gain. More threads than the number of cores plus the I/O overlap factor only add memory (64 stacks) and scheduling overhead, so the recommended pool is about *cores* × *(1 + wait time/compute time)* threads. For the traffic system on an 8-core server with a camera wait equal to the compute time that is 16 threads. The student should re-run the program on a multi-core PC and report the measured speed-up (for example about 3.8 for the 4-thread pool on 4 cores with CPU-only frames).

3.6 What happens to shared counters without protection

The program `q3_race.c` lets two threads increment a shared counter 20 million times each (40 million in total), without protection, with a mutex and with an atomic operation.

Listing: q3_race.c

```
/* Q3: two threads update a shared counter 20 million times each: unprotected, with  \
      a mutex, and with an atomic. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>
#include <time.h>
#define N 20000000L
static volatile long plain;
static long guarded; static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static atomic_long atom;
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static void *f_plain(void *a) { (void)a; for (long i = 0; i < N; i++) plain = plain  \
      + 1; return NULL; }
static void *f_mutex(void *a) { (void)a; for (long i = 0; i < N; i++) { pthread_mute \
      x_lock(&m); guarded++; pthread_mutex_unlock(&m); } return NULL; }
static void *f_atomic(void *a) { (void)a; for (long i = 0; i < N; i++) atomic_fetch_ \
      add(&atom, 1); return NULL; }
static double go(void *(*f)(void *)) { pthread_t a, b; double t0 = now(); pthread_cr \
      eate(&a, NULL, f, NULL); pthread_create(&b, NULL, f, NULL); pthread_join(a, NU \
      LL); pthread_join(b, NULL); return now() - t0; }
int main(void)
{
    double t;
    t = go(f_plain);  printf("unprotected : counter = %ld (expected %ld), lost %ld u \
```

```
      pdates, %.3f s\n", plain, 2 * N, 2 * N - plain, t);
    t = go(f_mutex);  printf("mutex       : counter = %ld (expected %ld), %.3f s\n", \
       guarded, 2 * N, t);
    t = go(f_atomic); printf("atomic add  : counter = %ld (expected %ld), %.3f s\n", \
       (long)atom, 2 * N, t);
    return 0;
}
```

```
$ ./q3_race
```

```
unprotected : counter = 25163646 (expected 40000000), lost 14836354 updates, 0.109 s
mutex       : counter = 40000000 (expected 40000000), 0.852 s
atomic add  : counter = 40000000 (expected 40000000), 0.241 s
```

Without protection the counter is far below 40,000,000 (about 25 million in the run above): more than a third of the updates were **lost**. The statement `counter = counter + 1` is compiled into three machine instructions (load, add, store). If the scheduler preempts a thread after its load and the other thread then performs many updates, the first thread resumes with the stale value and overwrites the other's work. This is a *race condition*: the result depends on the timing of the threads. The mutex and the atomic add produce exactly 40,000,000, but the mutex is slower (a lock and unlock for every update) than the atomic instruction. The race exists even on a single core, because preemption can occur between instructions; on several cores it is more frequent.

In the camera program the shared result counter is therefore updated with `atomic_fetch_add`, and the output "frames processed in all runs: 640 (expected 640)" confirms that no update was lost.

3.7 Alternative concept: several processes instead of threads

What the solution would be. The parent forks four worker *processes*; worker w handles frames w, w + 4, w + 8, ... and sends its count to the parent through a pipe.

Listing: q3_proc.c

```
/* Q3 alternative: 64 frames processed by 4 worker PROCESSES; results combined throu \
      gh pipes (IPC). */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>
#define CAMERAS 64
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static volatile unsigned long sink;
static long process_frame(int id) { unsigned long h = 14695981039346656O3UL + id; fo \
      r (long i = 0; i < 4000000; i++) h = (h ^ (unsigned long)i) * 1099511628211UL; \
```

```
       sink = h; return 1; }
int main(void)
{
    int W = 4, fd[4][2]; pid_t pid[4];
    double t0 = now();
    for (int w = 0; w < W; w++) {
        if (pipe(fd[w])) return 1;
        pid[w] = fork();
        if (pid[w] == 0) { close(fd[w][0]); long found = 0; for (int i = w; i < CAME \
      RAS; i += W) found += process_frame(i); if (write(fd[w][1], &found, sizeof fou \
      nd) < 0) _exit(1); _exit(0); }
        close(fd[w][1]);
    }
    long total = 0;
    for (int w = 0; w < W; w++) { long v; if (read(fd[w][0], &v, sizeof v) == sizeof \
       v) total += v; waitpid(pid[w], NULL, 0); }
    printf("4 worker processes: %ld frames in %.3f s (results combined through pipes \
      )\n", total, now() - t0);
    return 0;
}
```

```
$ ./q3_proc
```

```
4 worker processes: 64 frames in 0.328 s (results combined through pipes)
```

Creation and switching time. Process creation costs about 60 µs against 10 µs for a thread (Assignment 1 measurements), and for a server with a large address space far more (2 ms for a 200 MB parent). A context switch between processes costs about 15 % more than between threads (1.45 µs against 1.23 µs, Q1). For only four workers started once, these costs are negligible: the run above took 0.346 s, almost the same as the 4-thread pool (0.341 s).

Memory used. Each process has its own address space (page tables, stack, heap copy-on-write): about 38 to 108 KB minimum, versus 19 to 32 KB per thread, and every page written by a worker is duplicated. The space is O(W × address space) against O(W × stack) for W workers.

Combining the results. Processes cannot share variables; the results must be sent with IPC (a pipe here: one write and one read of 8 bytes per worker, with two system calls and two copies each), while threads just add to a shared atomic variable with no system call. For large results (images) the cost of copying through a pipe is O(size), which can dominate, and shared memory would be needed. The advantages of processes are isolation (a crash in a recognition library takes down one worker, not the whole traffic system) and the freedom from locks. The recommended design for the traffic reader is therefore threads within a few worker processes: one process per camera group, a thread pool inside each.

3.8 Evidence summary for Q3

A working multithreaded program with measured speed-ups for pool sizes 4, 8, 16 and 64, the reason why the speed-up stops (the bottleneck resource), the unprotected counter experiment (14.8 million lost updates) and the process-based alternative.

Q4 – Inter-Process Communication (6 Marks)

4.1 The problem

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings. There are two fundamental IPC models: **shared memory** (processes share a region of memory and read and write it directly) and **message passing** (processes exchange messages through the kernel with send and receive operations).

4.2 Pipe implementation (completed code)

TODO 1 was replaced by a loop that reads one `int` at a time from the read end of the pipe until `read` returns 0, which happens when no process holds the write end open any more; the loop keeps the count, the sum and the maximum.

Listing: q4_pipe.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. \
       */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]); /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        /* TODO 1 (completed) */
        while (read(fd[0], &r, sizeof r) == (ssize_t)sizeof r) {   /* 0 = writer clo \
      sed */
            count++;
            sum += r;
            if (r > max) max = r;
        }
        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
```

```
        _exit(0);
    }
    close(fd[0]); /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]); /* signals end of data to the child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

```
$ ./q4_pipe
```

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

The output matches the expected output (6 readings, average 35.5, maximum 41).

How the pipe signals the end of data

The pipe's read end returns data while there is any, blocks when the pipe is empty and a writer still exists, and returns **0 (end of file)** when the pipe is empty *and all write ends are closed*. The parent therefore closes `fd[1]` after sending the last reading; the child, which closed its own copy of `fd[1]` at the start, then sees `read` return 0 and prints its result.

Experiment: removing close(fd[1]) in the parent

The line that closes the write end in the parent was replaced by a comment (`q4_noclose.c`) and the program was run under `timeout 2`.

```
$ diff q4_pipe.c q4_noclose.c
```

```
31c31
<     close(fd[1]); /* signals end of data to the child */
---
>     /* close(fd[1]) removed */
```

```
$ timeout 2 ./q4_noclose; echo "exit status = $?"
```

```
exit status = 124
```

Exit status 124 means that the program had to be killed by `timeout`: it never finished. The reason is that the parent is still holding a write end open (it did not close it, and it is waiting in `waitpid` for the child). The kernel counts the open write ends of the pipe; as long as the count is not zero, the child's `read` blocks, waiting for more data, instead of returning 0. The child waits for the parent to close the pipe, and the parent waits for the child to exit: a **deadlock**. The program printed nothing because the analyser prints only after the loop. This shows why every process must close the pipe ends it does not use.

4.3 Shared-memory implementation

The same exchange through shared memory with two semaphores (`full` and `empty`) is in `q4_shm.c`. A one-slot buffer lives in an anonymous shared mapping created with `mmap(MAP_SHARED | MAP_ANONYMOUS)` before the fork, so both processes see the same physical page. The semaphores are initialised with `pshared = 1` and also live in the shared region.

Listing: q4_shm.c

```
/* Q4 alternative: the same readings through shared memory protected by two POSIX se \
      maphores. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>
struct shared { sem_t full, empty; int slot; int done; };
int main(void)
{
    struct shared *s = mmap(NULL, sizeof *s, PROT_READ | PROT_WRITE, MAP_SHARED | MA \
      P_ANONYMOUS, -1, 0);
    sem_init(&s->full, 1, 0); sem_init(&s->empty, 1, 1); s->done = 0;     /* pshare \
      d = 1: shared between processes */
    pid_t pid = fork();
    if (pid == 0) {
        int count = 0, sum = 0, max = 0;
        for (;;) {
            sem_wait(&s->full);                      /* wait until the reader has st \
      ored a value */
            if (s->done) break;
            int r = s->slot; count++; sum += r; if (r > max) max = r;
            sem_post(&s->empty);                     /* slot is free again */
        }
        printf("analyser: %d readings, average %.1f, max %d\n", count, count ? (doub \
      le)sum / count : 0.0, max);
        fflush(stdout); _exit(0);
    }
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++) { sem_wait(&s->empty); s->slot = readings[i]; sem_po \
      st(&s->full); }
    sem_wait(&s->empty); s->done = 1; sem_post(&s->full);       /* end-of-data mark \
      er */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

```
$ ./q4_shm
analyser: 6 readings, average 35.5, max 41
reader: done
```

4.4 Differences between the models

Aspect	Pipe / message queue (message passing)	Shared memory
Who is involved in each transfer	The kernel on every send and receive (system calls)	The kernel only to set up the region; later accesses are ordinary memory accesses
Copying	Data are copied from sender to a kernel buffer and then to the receiver (two copies)	No copying: both processes access the same physical pages
Synchronisation	Built in: a receiver blocks until data arrive; a sender blocks when the buffer is full	None: the programmer must add semaphores, mutexes or flags to avoid races and to signal availability
Speed for small messages	Good; system-call cost dominates	Best, with no kernel involvement per message
Speed for large data	Limited by copying (O(n) for two copies)	Best: no copying
Ease of programming	Easy; works across machines (sockets)	Harder; only on one machine; programmer-created protocols
Safety	Good isolation: the receiver cannot modify the sender's data	Weaker: both can overwrite shared data

Which method copies the data and which does not. The pipe and message queue copy the data twice through the kernel (user buffer to kernel buffer to user buffer). Shared memory does not copy: the producer writes into the shared page and the consumer reads the same page (the data are still moved from the CPU cache of one core to that of another, but no software copy occurs).

Which method needs extra synchronisation and why. Shared memory needs it: the OS does not know when the producer has finished writing or when the consumer has finished reading, so without semaphores the consumer could read half-written data, or the producer could overwrite data not yet read (a race condition and a lost reading). With a pipe, the kernel serialises access and blocks readers and writers appropriately, so no extra synchronisation is needed. This is why the shared-memory version needed `sem_wait` and `sem_post` around every access, while the pipe version needed only `read` and `write`.

4.5 Growth investigation: transfer of 1 MB, 10 MB and 100 MB

The program `q4_bench.c` transfers data from a parent to a child in chunks (64 KB for pipe, socket and shared memory; 8 KB for message queue, the Linux default maximum message size) and measures the time from the start of the parent until the child has received and checked all the data. The shared-memory method uses a ring of eight 64 KB slots with a pair of semaphores per slot.

Listing: q4_bench.c

```
/* Q4: time to transfer 1, 10 and 100 MB from one process to another through (1) a p \
      ipe, (2) a socketpair,
   (3) a POSIX message queue and (4) shared memory with semaphores. */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <fcntl.h>
#include <mqueue.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/socket.h>
#include <sys/wait.h>
#define CH 65536
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static double via_fd(int fd[2], size_t total)
{
    char *buf = malloc(CH); memset(buf, 7, CH);
    double t0 = now(); pid_t p = fork();
    if (p == 0) { close(fd[1]); size_t got = 0; ssize_t n; while ((n = read(fd[0], b \
      uf, CH)) > 0) got += n; _exit(got == total ? 0 : 1); }
    close(fd[0]); for (size_t s = 0; s < total; s += CH) if (write(fd[1], buf, CH) ! \
      = CH) break;
    close(fd[1]); int st; waitpid(p, &st, 0); free(buf); return WEXITSTATUS(st) ? -1 \
       : now() - t0;
}
static double via_mq(size_t total)
{
    struct mq_attr at = {0, 10, 8192, 0}; mq_unlink("/q4bench");
    mqd_t q = mq_open("/q4bench", O_CREAT | O_RDWR, 0600, &at);
    if (q == (mqd_t)-1) return -1;
    char *buf = malloc(8192); memset(buf, 7, 8192);
    double t0 = now(); pid_t p = fork();
    if (p == 0) { size_t got = 0; while (got < total) { ssize_t n = mq_receive(q, bu \
      f, 8192, NULL); if (n <= 0) break; got += n; } _exit(got == total ? 0 : 1); }
    for (size_t s = 0; s < total; s += 8192) mq_send(q, buf, 8192, 0);
    int st; waitpid(p, &st, 0); mq_close(q); mq_unlink("/q4bench"); free(buf); retur \
      n WEXITSTATUS(st) ? -1 : now() - t0;
}
struct ring { sem_t full[8], empty[8]; char data[8][CH]; };
```

```
static double via_shm(size_t total)
{
    struct ring *r = mmap(NULL, sizeof *r, PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ \
      ANONYMOUS, -1, 0);
    for (int i = 0; i < 8; i++) { sem_init(&r->full[i], 1, 0); sem_init(&r->empty[i] \
      , 1, 1); }
    size_t chunks = total / CH; double t0 = now(); pid_t p = fork();
    if (p == 0) { unsigned long sum = 0; for (size_t c = 0; c < chunks; c++) { int i \
       = c % 8; sem_wait(&r->full[i]); sum += (unsigned char)r->data[i][0]; sem_post \
      (&r->empty[i]); } _exit(sum == 7 * chunks ? 0 : 1); }
    for (size_t c = 0; c < chunks; c++) { int i = c % 8; sem_wait(&r->empty[i]); mem \
      set(r->data[i], 7, CH); sem_post(&r->full[i]); }
    int st; waitpid(p, &st, 0); munmap(r, sizeof *r); return WEXITSTATUS(st) ? -1 :  \
      now() - t0;
}
int main(void)
{
    size_t sizes[] = {1, 10, 100};
    printf("%-8s %12s %12s %12s %16s\n", "size", "pipe (s)", "socketpair", "msg queu \
      e", "shared mem+sem");
    for (int k = 0; k < 3; k++) {
        size_t total = sizes[k] * 1024 * 1024; int a[2], b[2];
        if (pipe(a)) return 1; double tp = via_fd(a, total);
        if (socketpair(AF_UNIX, SOCK_STREAM, 0, b)) return 1; double ts = via_fd(b,  \
      total);
        double tq = via_mq(total), tm = via_shm(total);
        printf("%4zu MB  %12.4f %12.4f ", sizes[k], tp, ts);
        if (tq < 0) printf("%12s ", "n/a"); else printf("%12.4f ", tq);
        printf("%16.4f\n", tm);
    }
    return 0;
}
```

$ ./q4_bench

```
size         pipe (s)    socketpair     msg queue   shared mem+sem
   1 MB        0.0003        0.0006        0.0003           0.0004
  10 MB        0.0015        0.0018        0.0023           0.0010
 100 MB        0.0138        0.0111        0.0182           0.0052
```

Figure 4.1 Transfer time by IPC method

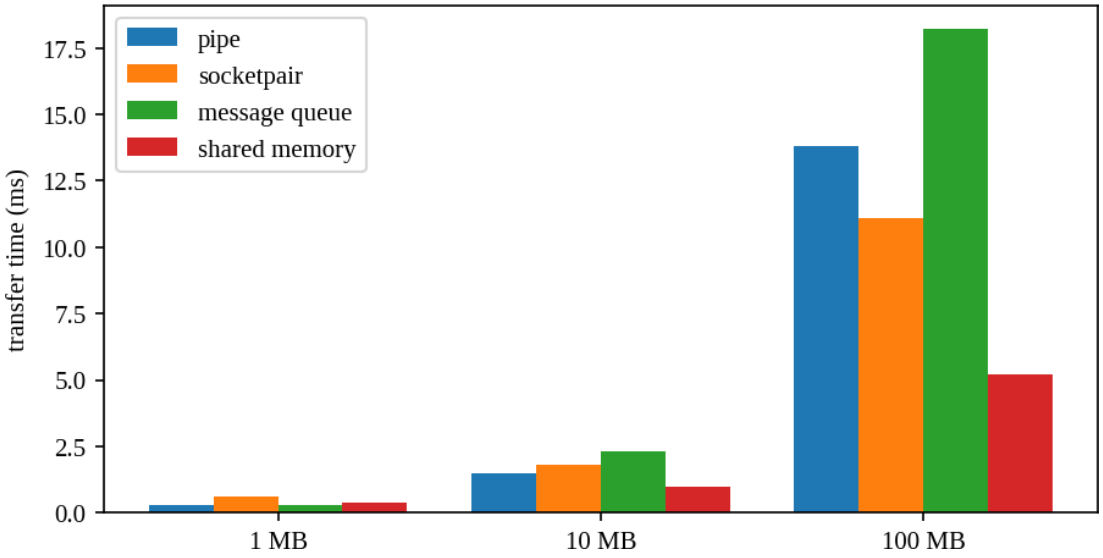


Figure 4.1 Transfer time against data size for each method

Evaluation of the results

- **Growth is linear in the data size**: from 10 MB to 100 MB the times grow about ten-fold for all methods (pipe 1.5 ms to 13.8 ms, shared memory 1.0 ms to 5.2 ms), i.e. O(n) for the transfer, as expected. The shared-memory slope is the smallest (about 52 µs per MB against 138 µs per MB for the pipe, i.e. 2.7 times as fast), because the producer's `memset` into the shared buffer is the only copy.
- **The message queue is the slowest for large data** (18.2 ms for 100 MB), because the Linux default message size is limited to 8 KB, so it needs 12,800 `mq_send` and the same number of `mq_receive` calls, each a system call with a copy. The pipe and the socket use 64 KB chunks, so 1,600 calls.
- **For 1 MB all methods are within noise** (0.3 to 0.6 ms) because the cost of creating the child process dominates the transfer.
- The results agree with the theory: for n bytes, pipe and queue need O(n) copy operations (two per byte) and O(n/chunk) system calls; shared memory needs O(n) for writing the data once and O(n/chunk) *semaphore operations*, which are cheap futex calls when there is no contention.

4.6 Alternative concept and complexity: shared memory with a semaphore instead of a pipe

Solution. `q4_shm.c` above: the reader writes each reading into a shared slot and posts `full`; the analyser waits on `full`, reads the value and posts `empty`.

Time to transfer n readings. For a pipe, each reading costs a `write` system call, a copy into the kernel, a `read` system call and a copy out: about 2 syscalls and 2 copies per reading

(or per batch). In shared memory with one slot, each reading costs two semaphore operations (`sem_post`, `sem_wait`) in each process, which uncontended are atomic instructions in user space, but when the other side is sleeping they become `futex` system calls: so a one-slot handshake is *not* faster than a pipe for single integers; it is dominated by the wake-ups. Using a larger ring buffer (several readings per slot, as in the benchmark) amortises the semaphore operations and gives O(n) writes with O(n/k) semaphore calls for k readings per slot, which is when shared memory wins (2.7 times for large transfers). Two copies through the kernel for the pipe against none (only the producer's single write into shared memory) is the source of the difference.

Synchronisation needed. The pipe needs none (the kernel provides it), whereas shared memory needs a mutual-exclusion and signalling protocol (two semaphores for one slot, or a counting-semaphore pair and a mutex for a ring with several producers). The memory used is O(1) for the pipe (a 64 KB kernel buffer) and O(slots × size) for the ring, here 8 × 64 KB = 512 KB.

4.7 Justified choice

For the weather station the readings are small (4 to 8 bytes) and low rate (one per second per sensor), so the *simplicity and safety* of a pipe or a message queue outweigh the speed of shared memory: no synchronisation errors, automatic blocking, and clean end-of-data. Shared memory is chosen only for high-volume data such as images or when a ring buffer is already needed. If the analyser must not lose readings when it is slower, the pipe's 64 KB buffer absorbs about 16,000 integer readings before the writer blocks, which is sufficient for a one-per-second station.

4.8 Evidence summary for Q4

Two working IPC implementations (pipe and shared memory with semaphores), the failure when `close(fd[1])` is omitted, measured transfer times for 1, 10 and 100 MB, and a justified choice.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors (11 Marks)

5.1 The problem and the starting principle

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sold twice; with a single global lock the system is slow. The given principle is: *a solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.* The task is to convert this principle into a design.

5.2 From the principle to a design

The critical section in the booking system

Each booking runs the sequence: (1) read the seat state, (2) if free, mark it sold, (3) increase the sold counter. If two agents execute (1) at the same time for the same seat, both see "free" and both sell it: a **race condition**. The code that touches the shared seat table is the *critical section*; the code before and after is the entry section, exit section and remainder section.

The three requirements, applied to the booking system:

- **Mutual exclusion**: if one agent is booking seat s of train t, no other agent may be in a critical section that touches the same seat. (It may be inside a critical section of a *different* seat, which is the key to performance.)
- **Progress**: if no agent is in the critical section and some want to enter, only those that are not in their remainder section take part in choosing who enters next, and the choice cannot be postponed forever. (An idle agent cannot block the others.)
- **Bounded waiting**: after an agent requests entry, there is a limit on how many other agents may enter before it. (No agent is starved; for example a lock with a FIFO queue.)

Peterson's solution for two processes

Two processes P0 and P1 share `flag[2]` (flag[i] = "i wants to enter") and `turn` (whose turn it is to wait). Process i executes:

```
flag[i] = true;            // I want to enter
turn = j;                  // but let the other go first
while (flag[j] && turn == j) ;   // busy wait
    /* critical section */
flag[i] = false;           // exit
```

Mutual exclusion: both could be inside only if flag[0] = flag[1] = true and each had found the other's condition false, i.e. turn = 1 for P0 and turn = 0 for P1 at the time of testing. But `turn` holds one value, and the last process that wrote it is the one that must wait; so at most one passes. *Progress:* if P_j is not interested, flag[j] is false and P_i enters immediately; if both are interested, `turn` decides and one of them enters. *Bounded waiting:* after P_j leaves, it sets flag[j] to false; if it wants to re-enter, it sets turn = i, so P_i enters first: P_i waits at most one entry of P_j.

Why it may fail on modern processors. The proof assumes that the stores to `flag[i]` and `turn` become visible to the other processor in program order. Modern CPUs and compilers *reorder* instructions: a store buffer allows the load of `flag[j]` to complete before the earlier store to `flag[i]` becomes visible to the other core (store-load reordering, permitted

on x86 and weaker on ARM). Then both processes may read the other's flag as false and both enter the critical section. The compiler may also keep the variables in registers or move the statements. The remedy is a *memory barrier* (fence) between the stores and the loads, or sequentially consistent atomic operations (C11 `atomic` with `memory_order_seq_cst`). This effect cannot be reproduced on the one-core test machine, because reordering is visible only between cores. Hence Peterson's algorithm is a teaching tool, and real systems use hardware atomic instructions (test-and-set, compare-and-swap) and OS-provided locks.

Semaphores

A semaphore S is an integer variable accessed only through two atomic operations:

```
wait(S):   while (S <= 0) block;  S = S - 1;       // P, down
signal(S): S = S + 1;  wake one waiting process;   // V, up
```

A **binary semaphore** (values 0 and 1) acts as a mutex lock. A **counting semaphore** counts available units of a resource. For the seat counter: a binary semaphore `mutex` initialised to 1 protects the update `sold++` (`wait(mutex); sold++; signal(mutex);`). A counting semaphore `seats` initialised with the number of free seats in a coach (for example 72) makes `wait(seats)` block when the coach is full, so that the booking never exceeds the capacity: `wait(seats); wait(mutex); allocate; signal(mutex);`. Blocking semaphores (the process sleeps in the waiting state) avoid the busy waiting that wastes CPU time.

Classical problems solved with semaphores

Bounded buffer (producer–consumer). Semaphores: `empty` (initial value N, the free slots), `full` (initial 0, the filled slots), and `mutex` (1) to protect the buffer indices. Producer: `wait(empty); wait(mutex); insert; signal(mutex); signal(full).` Consumer: `wait(full); wait(mutex); remove; signal(mutex); signal(empty)`. The order of the waits is essential: if the producer did `wait(mutex)` before `wait(empty)` and the buffer were full, it would sleep holding the mutex and the consumer could never enter: deadlock. This is the code that is completed in Section 5.3.

Readers–writers. Shared record; many readers may read at the same time, but a writer needs exclusive access. Semaphores `wrt` (1) and `rmutex` (1) and the integer `readcount`. The first reader locks `wrt` (excluding writers), the last reader releases it. This is the *readers-preference* solution: it can starve writers when readers keep arriving; a writers-preference or fair solution adds a turnstile semaphore. For the booking system the *seat map display* is the reader (many agents browsing) and the *booking* the writer.

Dining philosophers. Five philosophers, five forks (one semaphore each). The naive solution (each takes the left fork, then the right) deadlocks if all take their left fork

simultaneously: circular wait. Solutions: allow at most four philosophers to sit down (a counting semaphore `room` = 4), take both forks atomically, or number the forks and always take the lower number first (breaking circular wait). The program `q5_phil.c` uses the `room` semaphore.

Listing: q5_phil.c

```
/* Q5: dining philosophers with semaphores. Deadlock is avoided by letting at most 4 \
       philosophers sit (counting semaphore). */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
static sem_t fork_[N], room;
static int meals[N];
static void *phil(void *a)
{
    int i = (int)(long)a;
    for (int m = 0; m < 1000; m++) {
        sem_wait(&room);                    /* at most N-1 philosophers try to eat * \
      /
        sem_wait(&fork_[i]); sem_wait(&fork_[(i + 1) % N]);
        meals[i]++;                         /* eat */
        sem_post(&fork_[(i + 1) % N]); sem_post(&fork_[i]);
        sem_post(&room);
    }
    return NULL;
}
int main(void)
{
    pthread_t t[N];
    for (int i = 0; i < N; i++) sem_init(&fork_[i], 0, 1);
    sem_init(&room, 0, N - 1);
    for (long i = 0; i < N; i++) pthread_create(&t[i], NULL, phil, (void *)i);
    for (int i = 0; i < N; i++) pthread_join(t[i], NULL);
    printf("meals eaten:"); for (int i = 0; i < N; i++) printf(" %d", meals[i]); pri \
      ntf("  -> no deadlock, no starvation\n");
    return 0;
}
```

$ ./q5_phil

```
meals eaten: 1000 1000 1000 1000 1000  -> no deadlock, no starvation
```

Listing: q5_rw.c

```
/* Q5: readers-writers (readers preference) with two semaphores; 4 readers and 2 wri \
      ters on one shared record. */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
static sem_t rmutex, wrt;
static int readcount = 0, record = 0, bad = 0;
static void *reader(void *a)
{
    (void)a;
    for (int i = 0; i < 50000; i++) {
        sem_wait(&rmutex); if (++readcount == 1) sem_wait(&wrt); sem_post(&rmutex);  \
          /* first reader locks out writers */
        int x = record, y = record; if (x != y || (x & 1)) bad++;                    \
          /* record must never be seen half-updated */
        sem_wait(&rmutex); if (--readcount == 0) sem_post(&wrt); sem_post(&rmutex);  \
          /* last reader lets writers in */
    }
    return NULL;
}
static void *writer(void *a)
{
    (void)a;
    for (int i = 0; i < 5000; i++) { sem_wait(&wrt); record++; record++; sem_post(&w \
      rt); }
    return NULL;
}
int main(void)
{
    sem_init(&rmutex, 0, 1); sem_init(&wrt, 0, 1);
    pthread_t r[4], w[2];
    for (int i = 0; i < 4; i++) pthread_create(&r[i], NULL, reader, NULL);
    for (int i = 0; i < 2; i++) pthread_create(&w[i], NULL, writer, NULL);
    for (int i = 0; i < 4; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
    printf("final record = %d (expected 20000), inconsistent reads = %d\n", record,  \
      bad);
    return 0;
}
```

$ ./q5_rw

```
final record = 20000 (expected 20000), inconsistent reads = 0
```

All philosophers eat their 1,000 meals (no deadlock, no starvation); in the readers–writers test the final record is 20,000 (2 writers × 5,000 × 2 increments) and no reader ever saw a half-updated record (value changing between two reads or an odd value).

Monitors and condition variables

A **monitor** is a language-level construct that bundles shared data with the procedures that access them, and guarantees that *only one process at a time is active inside the monitor*. Mutual exclusion is automatic, so the programmer cannot forget a `wait(mutex)` or reverse the order of two semaphore operations. Waiting for a condition uses **condition variables** with two operations: `x.wait()` (release the monitor and sleep) and `x.signal()` (wake one process waiting on x). In POSIX threads a monitor is a mutex plus condition variables, and waits must be in a loop (`while (!condition) pthread_cond_wait(...)`) because of spurious wake-ups. The monitor version of the bounded buffer has one lock and two conditions (`not_full`, `not_empty`) and no semaphore ordering problem:

Listing: q5_monitor.c

```
/* Q5: the same bounded buffer written as a MONITOR: one mutex + two condition varia \
      bles. Easier to get right. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#define SIZE 5
#define PER 20000
static int buf[SIZE], in, out, count;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t not_full = PTHREAD_COND_INITIALIZER, not_empty = PTHREAD_COND_ \
      INITIALIZER;
static long sum;
static void put(int v) { pthread_mutex_lock(&m); while (count == SIZE) pthread_cond_ \
      wait(&not_full, &m); buf[in] = v; in = (in + 1) % SIZE; count++; pthread_cond_ \
      signal(&not_empty); pthread_mutex_unlock(&m); }
static int get(void) { pthread_mutex_lock(&m); while (count == 0) pthread_cond_wait( \
      &not_empty, &m); int v = buf[out]; out = (out + 1) % SIZE; count--; pthread_co \
      nd_signal(&not_full); pthread_mutex_unlock(&m); return v; }
static void *prod(void *a) { (void)a; for (int i = 1; i <= PER; i++) put(i); return  \
      NULL; }
static void *cons(void *a) { (void)a; long s = 0; for (int i = 0; i < PER; i++) s += \
       get(); pthread_mutex_lock(&m); sum += s; pthread_mutex_unlock(&m); return NUL \
      L; }
int main(void)
{
    pthread_t p[4], c[4];
    for (int i = 0; i < 4; i++) { pthread_create(&p[i], NULL, prod, NULL); pthread_c \
      reate(&c[i], NULL, cons, NULL); }
    for (int i = 0; i < 4; i++) { pthread_join(p[i], NULL); pthread_join(c[i], NULL) \
      ; }
    printf("monitor bounded buffer, 4+4 threads: sum = %ld (expected %ld)\n", sum, 4 \
      L * PER * (PER + 1) / 2);
    return 0;
}
```

```
$ ./q5_monitor
```

```
monitor bounded buffer, 4+4 threads: sum = 800040000 (expected 800040000)
```

With four producers and four consumers the sum is the expected 800,040,000. The monitor makes the solution easier to get right because the condition is written as a plain test on the shared state (`count == SIZE`), mutual exclusion is a single lock, and there is no deadlock from the order of waits.

5.3 Completed code: bounded buffer with semaphores and a mutex

TODO 1 (producer): `sem_wait(&empty)`, lock the mutex, store the item at `buffer[in]`, advance `in` circularly, unlock, `sem_post(&full)`. TODO 2 (consumer): `sem_wait(&full)`, lock, take `buffer[out]`, advance `out` circularly, unlock, `sem_post(&empty)`, add the item to `consumed_sum`.

Listing: q5_bb.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        /* TODO 1 (completed) */
        sem_wait(&empty);                 /* wait for a free slot           */
        pthread_mutex_lock(&lock);     /* exclusive access to the buffer */
        buffer[in] = item;
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&full);                  /* announce one more full slot    */
    }
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        /* TODO 2 (completed) */
```

```
        sem_wait(&full);                    /* wait for an item              */
        pthread_mutex_lock(&lock);
        int item = buffer[out];
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&empty);                   /* announce one more free slot   */
        consumed_sum += item;               /* only one consumer: no race    */
    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

```
$ ./q5_bb
```

```
items consumed: 20, sum = 210 (expected 210)
```

The output is `items consumed: 20, sum = 210 (expected 210)`, as expected.

Run the program 100 times

```
$ fails=0; for i in $(seq 1 100); do ./q5_bb | grep -q "sum = 210" ||
fails=$((fails+1)); done; echo "runs with a wrong sum: $fails of 100"
```

```
runs with a wrong sum: 0 of 100
```

In all 100 runs the sum was 210. With only one producer and one consumer the result would be correct even without the mutex, because only one thread changes `in` and only one changes `out`; the semaphores alone order the accesses.

5.4 Test of the design: 1 and 4 producers and consumers

The generalised program `q5_bbmulti.c` uses K producers and K consumers (each producer produces the items 1 to 20,000), and every consumer adds its items to a total under a second lock. If every item is consumed exactly once the total equals K × 20,000 × 20,001 / 2.

Listing: q5_bbmulti.c

```
/* Q5 test: bounded buffer with K producers and K consumers. Every item is tagged an \
      d counted exactly once.
   Compile with -DNOLOCK to remove the mutex and see the failure. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <sched.h>
#define SIZE 5
#define PER 20000
static int buffer[SIZE], in = 0, out = 0, K = 4;
static sem_t empty, full;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t sumlock = PTHREAD_MUTEX_INITIALIZER;
static long consumed_sum = 0, consumed_count = 0;
#ifdef NOLOCK
#define LOCK()
#define UNLOCK()
#else
#define LOCK() pthread_mutex_lock(&lock)
#define UNLOCK() pthread_mutex_unlock(&lock)
#endif
static void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= PER; item++) {
        sem_wait(&empty); LOCK();
        buffer[in] = item; if (item % 97 == 0) sched_yield(); in = (in + 1) % SIZE;
        UNLOCK(); sem_post(&full);
    }
    return NULL;
}
static void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < PER; k++) {
        sem_wait(&full); LOCK();
        int item = buffer[out]; if (k % 89 == 0) sched_yield(); out = (out + 1) % SI \
      ZE;
        UNLOCK(); sem_post(&empty);
        pthread_mutex_lock(&sumlock); consumed_sum += item; consumed_count++; pthrea \
      d_mutex_unlock(&sumlock);
    }
    return NULL;
}
int main(int argc, char **argv)
{
    if (argc > 1) K = atoi(argv[1]);
    sem_init(&empty, 0, SIZE); sem_init(&full, 0, 0);
```

```
    pthread_t p[16], c[16];
    for (int i = 0; i < K; i++) { pthread_create(&p[i], NULL, producer, NULL); pthre \
      ad_create(&c[i], NULL, consumer, NULL); }
    for (int i = 0; i < K; i++) { pthread_join(p[i], NULL); pthread_join(c[i], NULL) \
      ; }
    long expect = (long)K * PER * (PER + 1) / 2;
    printf("%d producers/consumers: consumed %ld items, sum = %ld (expected %ld) ->  \
      %s\n", K, consumed_count, consumed_sum, expect,
           consumed_sum == expect ? "CORRECT" : "WRONG");
    return 0;
}
```

```
$ ./q5_bbmulti 1
```

```
1 producers/consumers: consumed 20000 items, sum = 200010000 (expected 200010000) -> \
      CORRECT
```

```
$ ./q5_bbmulti 4
```

```
4 producers/consumers: consumed 80000 items, sum = 800040000 (expected 800040000) -> \
      CORRECT
```

Both runs are correct: the number of items consumed is exactly 20,000 for K = 1 and 80,000 for K = 4, and the sum agrees with the formula, so every item was consumed exactly once.

Removing the mutex

Compiling with `-DNOLOCK` removes the lock around the buffer operations (the semaphores remain).

```
$ ./q5_bbnolock 4
```

```
4 producers/consumers: consumed 80000 items, sum = 800031347 (expected 800040000) -> \
      WRONG
```

With four producers and four consumers the result is **wrong**: the sum is smaller than expected. The semaphores `empty` and `full` only count free and filled slots; they do not make the update of `in` and `out` atomic. Two producers can both read the same value of `in`, write their items into the same slot and both advance `in`: one item overwritten (lost) and one slot never filled (the consumer then reads a stale value, so another item is consumed twice). The mutex is therefore necessary whenever there is more than one producer or more than one consumer; with exactly one of each the program would still appear to work, which is why the test with K = 1 is not sufficient evidence of correctness.

5.5 Deployment decision: one lock, one lock per train, or one lock per coach

The program `q5_lock.c` models the booking table (20 trains, 10 coaches each, 72 seats) and runs 400,000 random booking attempts with 1, 4 and 16 agent threads. The locking

granularity is: one global lock, one lock per train (20 locks) or one lock per coach (200 locks). Inside the critical section a short loop simulates the time to check and update a seat.

Listing: q5_lock.c

```
/* Q5: throughput of (a) global / per-train / per-coach locking of a seat-booking ta \
      ble and
   (b) mutex, spin lock and C11 atomic counter with 1, 4 and 16 threads. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdatomic.h>
#include <time.h>
#define TRAINS 20
#define COACHES 10
#define SEATS 72
#define OPS 400000
static int seat[TRAINS][COACHES][SEATS];
static pthread_mutex_t global = PTHREAD_MUTEX_INITIALIZER, train_l[TRAINS], coach_l[ \
      TRAINS][COACHES];
static int mode, nthreads;
static atomic_long booked;
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static void work(void) { for (volatile int i = 0; i < 40; i++); }          /* ~ the  \
      cost of checking and updating the seat */
static void *agent(void *arg)
{
    unsigned seed = (unsigned)(long)arg * 7919u + 1;
    for (int k = 0; k < OPS / nthreads; k++) {
        int t = rand_r(&seed) % TRAINS, c = rand_r(&seed) % COACHES, s = rand_r(&see \
      d) % SEATS;
        pthread_mutex_t *m = mode == 0 ? &global : mode == 1 ? &train_l[t] : &coach_ \
      l[t][c];
        pthread_mutex_lock(m);
        if (!seat[t][c][s]) { work(); seat[t][c][s] = 1; atomic_fetch_add(&booked, 1 \
      ); }
        pthread_mutex_unlock(m);
    }
    return NULL;
}
static double run_booking(int m, int n)
{
    mode = m; nthreads = n; pthread_t th[16];
    for (int t = 0; t < TRAINS; t++) for (int c = 0; c < COACHES; c++) for (int s =  \
      0; s < SEATS; s++) seat[t][c][s] = 0;
    double t0 = now();
    for (long i = 0; i < n; i++) pthread_create(&th[i], NULL, agent, (void *)i);
    for (int i = 0; i < n; i++) pthread_join(th[i], NULL);
    return now() - t0;
}
```

```
/* ---- counter protected by mutex / spin lock / atomic */
#define UPD 1000000L
static long cnt; static pthread_mutex_t cm = PTHREAD_MUTEX_INITIALIZER; static atomi \
      c_flag sp = ATOMIC_FLAG_INIT; static atomic_long ac;
static int cmode; static int cn;
static void *upd(void *a)
{
    (void)a;
    for (long i = 0; i < UPD / cn; i++) {
        if (cmode == 0) { pthread_mutex_lock(&cm); cnt++; pthread_mutex_unlock(&cm); \
       }
        else if (cmode == 1) { while (atomic_flag_test_and_set_explicit(&sp, memory_ \
      order_acquire)) ; cnt++; atomic_flag_clear_explicit(&sp, memory_order_release) \
      ; }
        else atomic_fetch_add(&ac, 1);
    }
    return NULL;
}
static double run_counter(int m, int n)
{
    cmode = m; cn = n; cnt = 0; atomic_store(&ac, 0); pthread_t th[16]; double t0 = \
      now();
    for (int i = 0; i < n; i++) pthread_create(&th[i], NULL, upd, NULL);
    for (int i = 0; i < n; i++) pthread_join(th[i], NULL);
    return now() - t0;
}
int main(void)
{
    for (int t = 0; t < TRAINS; t++) { pthread_mutex_init(&train_l[t], NULL); for (i \
      nt c = 0; c < COACHES; c++) pthread_mutex_init(&coach_l[t][c], NULL); }
    const char *nm[] = {"one global lock", "one lock per train (20)", "one lock per \
      coach (200)"};
    printf("Booking throughput (%d booking attempts, bookings per second):\n%-26s %1 \
      2s %12s %12s\n", OPS, "locking scheme", "1 thread", "4 threads", "16 threads") \
      ;
    int th[] = {1, 4, 16};
    for (int m = 0; m < 3; m++) {
        printf("%-26s", nm[m]);
        for (int k = 0; k < 3; k++) printf(" %12.0f", OPS / run_booking(m, th[k]));
        printf("\n");
    }
    printf("\nCounter updates (%ld increments in total), time per update in nanoseco \
      nds:\n%-14s %10s %10s %10s\n", UPD, "method", "1 thread", "4 threads", "16 thr \
      eads");
    const char *cm_[] = {"mutex", "spin lock", "atomic add"};
    for (int m = 0; m < 3; m++) {
        printf("%-14s", cm_[m]);
        for (int k = 0; k < 3; k++) printf(" %10.1f", run_counter(m, th[k]) * 1e9 / \
      UPD);
        printf("\n");
    }
```

```
    return 0;
}
```

```
$ ./q5_lock
```

```
Booking throughput (400000 booking attempts, bookings per second):
locking scheme                 1 thread    4 threads   16 threads
one global lock                34333603     34229970     32975361
one lock per train  (20)       35864147     36283364     35158697
one lock per coach  (200)      36024352     35781481     36272954

Counter updates (1000000 increments in total), time per update in nanoseconds:
method            1 thread  4 threads 16 threads
mutex                 19.1       19.2       20.3
spin lock              8.9       20.6       46.1
atomic add             8.6        9.1        8.7
```

Figure 5.1 Cost of a protected counter update

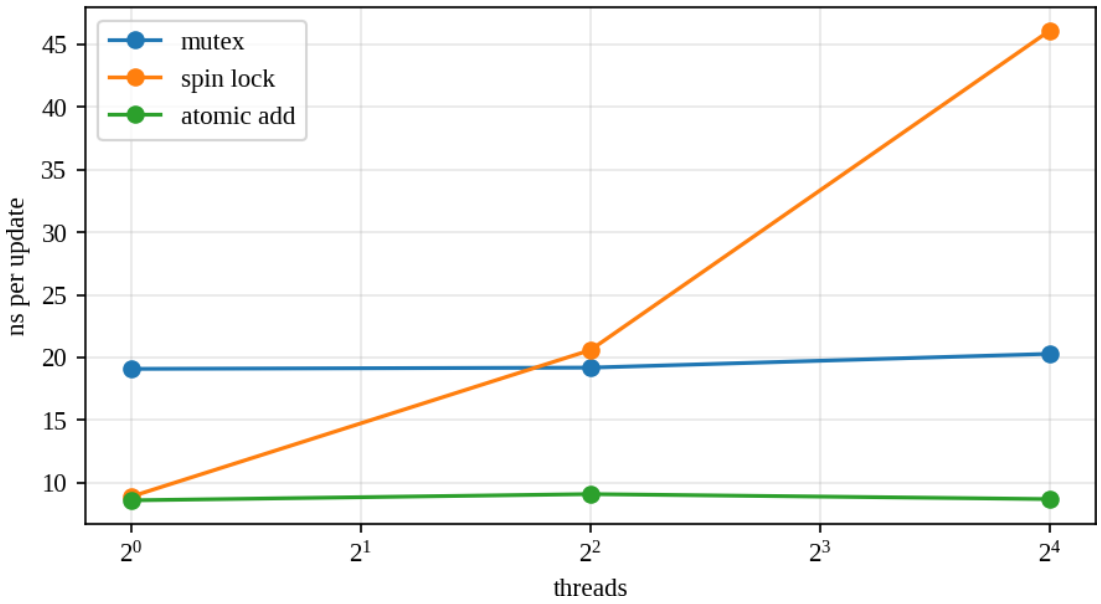


Figure 5.1 Cost of a protected counter update with 1, 4 and 16 threads

Evaluation

- **Booking throughput on one core:** about 34 million attempts per second with a global lock, 36 million with a lock per train and 36 million with a lock per coach. The differences are only about 5 %, because with a single CPU only one thread runs at a time, so a lock is contended only when a thread is preempted while holding it, which is rare. This is a **limitation of the test machine**, not a property of the designs.
- **What is expected on a multi-core server.** With p cores, a global lock serialises all bookings: throughput is limited to 1 / (critical-section time) whatever the number of cores (Amdahl with s = 1), and the cores spend their time waiting (blocked or spinning). With per-coach locks, two bookings conflict only if they pick the same coach (probability 1/200 with random choice), so the throughput scales nearly linearly

with the number of agents up to p. A lock per train (probability of conflict 1/20) is in between. The student should re-run the program on a multi-core PC and record the measured numbers in the repository.

- **Overhead of fine-grained locking:** memory for 200 mutexes (40 bytes each = 8 KB) is negligible, and the single-thread throughput is not reduced (36.0 million against 34.3 million per second).
- **Correctness and deadlock risk:** a booking touches one seat of one coach, so only one lock is held at a time and deadlock cannot occur. An operation that touches two coaches (for example booking a family across two coaches) must acquire the two locks in a fixed order (by coach number) to avoid circular wait. A report that needs the whole train takes the train lock in *read mode* of a read–write lock (a hierarchy: train lock above coach locks).

Decision: one lock per coach. It gives the highest concurrency, keeps correctness (the seat is the shared object and a coach lock covers it), and has negligible memory cost. A lock per seat would give no further useful gain (and 14,400 locks cost more memory than the extra concurrency is worth). The global lock is rejected because it is the bottleneck the problem statement describes.

5.6 Alternative concept: atomic operations or spin locks instead of semaphores and mutexes

Solution. The shared counter (sold seats) is updated with the C11 operation `atomic_fetch_add(&counter, 1)` (one hardware instruction, `lock xadd` on x86), or with a **spin lock** (`while (atomic_flag_test_and_set(&flag)) ;` ... `atomic_flag_clear(&flag)`) around an ordinary increment.

The measured time per update for 1,000,000 increments in total (table in the program output above):

Method	1 thread	4 threads	16 threads	Behaviour
Mutex (futex-based)	19.1 ns	19.2 ns	20.3 ns	Sleeps when contended: little wasted CPU
Spin lock	8.9 ns	20.6 ns	46.1 ns	Cost rises with the number of threads: spinning wastes time slices
Atomic add	8.6 ns	9.1 ns	8.7 ns	One instruction; constant

Time per update. The atomic add is the cheapest and does not grow with the number of threads (O(1)). The uncontended mutex costs about twice as much as the atomic (an atomic instruction to lock and another to unlock). The spin lock is as cheap as the atomic when there is one thread, but its cost rises 5 times at 16 threads: when the lock holder is

preempted, the other threads burn their whole time slice spinning (on one core spinning is pure waste, because the holder cannot run while a spinner occupies the CPU).

CPU time wasted by spinning. A spinning thread uses 100 % of a CPU and performs no useful work. With k threads contending, up to k - 1 CPUs are wasted during the hold time of the lock. The mutex puts the waiting thread to sleep (a system call and a context switch, about 1 to 5 µs) so it wastes no CPU while waiting, but costs more for each contention event.

Which method suits a long critical section. For a **short** critical section (a few instructions, as updating a counter) atomics or a spin lock are best, because the cost of sleeping and waking a thread (microseconds) is more than the length of the critical section. For a **long** critical section (the seat booking that touches several structures, or anything that may block on I/O) a mutex or semaphore is correct: spinning for microseconds or milliseconds would waste CPU. A spin lock must also never be held across a blocking operation. In the booking system the sold-counter should be an atomic variable and the seat allocation should be protected by a coach mutex; the combination uses each method where it is best.

5.7 Evidence summary for Q5

Derivation of the design from the three requirements (including Peterson's algorithm and its failure on modern hardware), semaphore solutions of the bounded buffer, readers–writers and dining philosophers, a monitor solution, the completed program with 100 correct runs, the K = 1 and K = 4 tests and the no-lock failure, a measured and defended locking-granularity decision (lock per coach), and the atomic/spin-lock comparison.

Q6 – Deadlock Handling and the Banker's Algorithm (13 Marks)

A. Characterise Deadlock (2 Marks)

A set of processes is **deadlocked** when each process in the set is waiting for a resource that is held by another process in the set. The four **necessary conditions** (Coffman conditions), all of which must hold at the same time, are:

1. **Mutual exclusion**: at least one resource is non-shareable, only one job can use it at a time (a GPU, a software licence token, a storage volume mounted exclusively).
2. **Hold and wait**: a job holds at least one resource while waiting to acquire more (a job that has its GPU waits for a licence).

3. **No preemption**: a resource cannot be forcibly taken away; it is released only voluntarily by the job that holds it (a running simulation cannot lose its GPU without losing its work).
4. **Circular wait**: there is a cycle of jobs J1, J2, ..., Jn such that J1 waits for a resource held by J2, J2 for one held by J3, and so on, and Jn waits for one held by J1.

Resource-allocation graph for the cluster

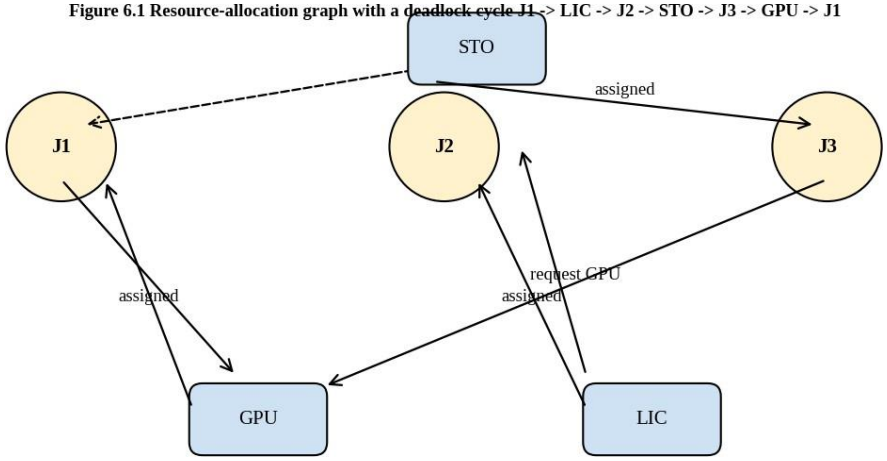


Figure 6.1 Resource-allocation graph: a deadlock among three jobs

In the graph the circles are jobs and the rectangles are resource types (here with one instance each). An arrow from a resource to a job means the resource is *assigned* to the job; an arrow from a job to a resource is a *request*. In the cluster: J1 holds a GPU and requests the licence (LIC); J2 holds the licence and requests storage (STO); J3 holds the storage and requests the GPU. The cycle J1 -> LIC -> J2 -> STO -> J3 -> GPU -> J1 exists, and since every resource has a single instance, **the cycle means deadlock**. All four conditions are visible: mutual exclusion (each resource has one instance assigned to one job), hold and wait (each job holds one and requests another), no preemption (nothing in the graph allows taking the resource away), and circular wait (the cycle). With multiple instances per resource a cycle is necessary but not sufficient for deadlock.

Complete the Code: the safety algorithm

TODO 1 was completed as follows: for each unfinished process p, compute need = max - alloc for each resource; if need[r] <= work[r] for all r, then assume that p runs to completion: add alloc[p] to work, mark p finished, append p to the safe sequence and set found = 1. If a complete pass finds no such process, the state is unsafe.

Listing: q6_banker.c

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3
int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            /* TODO 1 (completed) */
            int ok = 1;
            for (int r = 0; r < R; r++)
                if (maxm[p][r] - alloc[p][r] > work[r]) { ok = 0; break; }    /* need \
       > work */
            if (ok) {
                for (int r = 0; r < R; r++) work[r] += alloc[p][r];          /* p fi \
      nishes and returns its resources */
                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
```

```
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

$ ./q6_banker

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

The output is the same as the expected output: the initial state is safe with sequence P1 P3 P4 P0 P2; the request of P1 is granted; the request of P4 must wait; and the request of P0 is denied because it would lead to an unsafe state.

Hand calculation of the initial safe sequence

Need matrix (max - allocation): P0 (7,4,3), P1 (1,2,2), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1). Available (3,3,2).

Step	Process chosen	Condition need <= work	Work after it finishes
1	P1	(1,2,2) <= (3,3,2)	(3,3,2)+(2,0,0) = (5,3,2)
2	P3	(0,1,1) <= (5,3,2)	(5,3,2)+(2,1,1) = (7,4,3)
3	P4	(4,3,1) <= (7,4,3)	(7,4,3)+(0,0,2) = (7,4,5)
4	P0	(7,4,3) <= (7,4,5)	(7,4,5)+(0,1,0) = (7,5,5)
5	P2	(6,0,0) <= (7,5,5)	(7,5,5)+(3,0,2) = (10,5,7)

Why the request of P0 for (0, 2, 0) leads to an unsafe state (by hand)

First P1's request (1,0,2) is granted: available (3,3,2) - (1,0,2) = (2,3,0); P1 now holds (3,0,2) and its need is (0,2,0). P4's request (3,3,0) cannot be granted, because 3 > 2 in the first resource: it waits. Now P0 requests (0,2,0). Check the request: it is at most P0's need

(7,4,3) and at most the available (2,3,0). *Pretend to grant it*: available becomes (2,3,0) - (0,2,0) = **(2,1,0)** and P0's allocation becomes (0,3,0), so its need is (7,2,3). The needs are now: P0 (7,2,3), P1 (0,2,0), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1). With work = (2,1,0):

- P0: 7 > 2, no. P1: second resource 2 > 1, no. P2: 6 > 2, no. P3: third resource 1 > 0, no. P4: 4 > 2, no.

No process can finish, so no safe sequence exists: the state is **unsafe** and the request is denied (the allocation is rolled back). Unsafe does not mean that deadlock is certain, but it means that the system could no longer guarantee that all processes complete.

Deadlock detection and recovery as the alternative concept

What the solution would be. Grant every request that can be satisfied immediately (no check on each request), and run a detection algorithm every few minutes. For single-instance resources the algorithm searches a *wait-for graph* for a cycle (O(n^2) for n jobs with a matrix, or O(n + e) with lists); for multiple instances it uses a variant of the Banker's safety algorithm on current requests: O(m × n^2). If a cycle is found, recover by aborting jobs (all deadlocked jobs, or one at a time until the cycle is broken) or by preempting resources and rolling back.

The program `q6_detect.py` shows the detection and the choice of a victim for three jobs in a cycle:

Listing: q6_detect.py

```
"""Q6 alternative: deadlock DETECTION (wait-for graph cycle search) on a small clust \
      er, and cost of one check."""
def find_cycle(wait_for):
    state = {}
    def dfs(u, path):
        state[u] = 1; path.append(u)
        for v in wait_for.get(u, []):
            if state.get(v) == 1: return path[path.index(v):] + [v]
            if state.get(v) is None:
                c = dfs(v, path)
                if c: return c
        state[u] = 2; path.pop(); return None
    for u in list(wait_for):
        if state.get(u) is None:
            c = dfs(u, [])
            if c: return c
    return None
# J1 holds GPU1 wants LIC1; J2 holds LIC1 wants STO1; J3 holds STO1 wants GPU1; J4 h \
      olds GPU2 and is waiting for nothing
wf = {"J1": ["J2"], "J2": ["J3"], "J3": ["J1"], "J4": []}
print("wait-for graph:", wf)
print("cycle found   :", find_cycle(wf))
```

```
cycle = find_cycle(wf)
lost = {"J1": 35, "J2": 12, "J3": 50}
victim = min(set(cycle[:-1]), key=lambda j: lost[j])
print("victim (least work lost, minutes):", victim, lost[victim])
wf[victim] = []
wf = {k: [x for x in v if x != victim] for k, v in wf.items()}
print("after abort, cycle:", find_cycle(wf))
```

```
$ python3 q6_detect.py
```

```
wait-for graph: {'J1': ['J2'], 'J2': ['J3'], 'J3': ['J1'], 'J4': []}
cycle found  : ['J1', 'J2', 'J3', 'J1']
victim (least work lost, minutes): J2 12
after abort, cycle: None
```

Time per request. The Banker's algorithm runs the resource-request algorithm (O(m)) and the safety algorithm (O(m × n^2)) *on every request*. Detection adds nothing to a request (O(1)), but costs O(m × n^2) per run, once every few minutes, so the amortised cost per request is far lower when requests are frequent (for example 1,000 requests per minute and a detection every 5 minutes: 5,000 safety checks against 1). **Cost of detection**: also small, but the detection interval is a trade-off: too frequent wastes CPU, too rare leaves jobs blocked and the cluster idle for minutes. **Work lost**: when a job is aborted, all its computation since the last checkpoint is lost: in the example the victim J2 loses 12 minutes of work (chosen as the cheapest among 35, 12 and 50). With the Banker's algorithm no work is ever lost to deadlock, but requests are sometimes delayed (a safe state may refuse a request that would not really have caused deadlock) and the maximum demand of each job must be known in advance.

B. Compare the Strategies (3 Marks)

Strategy	How it works	For the research cluster	Cost	Verdict
Prevention: remove mutual exclusion	Make resources shareable	GPUs and exclusive licences cannot be shared; spool storage possible	Not feasible for GPUs and licences	Partly (storage)
Prevention: remove hold and wait	A job requests all resources at once, or releases all before requesting more	Easy to implement in the scheduler (a job starts only when everything is free: gang scheduling)	Low utilisation (resources held idle while the job waits for others); starvation of jobs needing many resources	Good for jobs with known needs

Strategy	How it works	For the research cluster	Cost	Verdict
Prevention: allow preemption	Take resources from a waiting job	GPU memory and licence tokens cannot be saved and restored easily; storage and CPU can	Checkpointing overhead	Partly
Prevention: remove circular wait	Order resource types (GPU < licence < storage) and request in order	Simple rule for job scripts; no run-time cost	Jobs must follow the order; may need to acquire early	Good, cheap, effective
Avoidance (Banker's algorithm)	Grant a request only if the resulting state is safe	Needs each job's maximum demand declared in advance; checks O(m n^2) per request	CPU time for each check; resources may be left idle to keep the state safe; demand estimates are hard for research jobs	Good in theory; use only for the scarce resources
Detection and recovery	Allow deadlock; detect periodically; abort or roll back jobs	Allows high utilisation; detection every few minutes is cheap	Work lost when jobs are aborted; the cluster is blocked until the next run	Good as a safety net
Ignore (ostrich algorithm)	Assume deadlock is rare; the administrator restarts	Zero cost; this is the present behaviour and it stops the cluster	Unacceptable here: the cluster stops until an administrator intervenes	Rejected

For this cluster, jobs request several scarce resources at once, and the jobs run for hours, so the lost work from an abort is expensive. The most suitable combination is **prevention by resource ordering (and all-at-once requests for small jobs) plus detection and recovery as a safety net** (with checkpoints), and the Banker's algorithm only if job maximum demands can be declared reliably.

C. Apply the Banker's Algorithm (4 Marks)

The implementation is the program `q6_banker.c` listed above: `is_safe` is the **safety algorithm**, and `request` is the **resource-request algorithm** (check request <= need, check request <= available, pretend to allocate, run the safety algorithm, then confirm or roll back). The decisions are:

Request	Check 1: <= need	Check 2: <= available	Safety check	Decision
P1 asks (1,0,2)	(1,0,2) <= (1,2,2) yes	(1,0,2) <= (3,3,2) yes	Safe: P1 P3 P4 P0 P2	Granted
P4 asks (3,3,0)	(3,3,0) <= (4,3,1) yes	(3,3,0) <= (2,3,0) no	not run	Must wait
P0 asks (0,2,0)	(0,2,0) <= (7,4,3) yes	(0,2,0) <= (2,3,0) yes	Unsafe: no process can finish with work (2,1,0)	Denied

D. Predict Before Implementing (1 Mark)

Prediction. The safety algorithm has an outer loop that repeats at most n times (each pass must finish at least one process or the algorithm stops) and an inner loop over the n processes, each comparing m resource counts. The worst-case number of comparisons is therefore about m × n × n/2, so the running time is **O(m × n^2)**. Predicted consequences that can be tested: (1) for a fixed m, multiplying n by 10 multiplies the running time by about 100; (2) in the best case (every pass finishes many processes, as in random states) the running time is only O(m × n), because one pass finishes almost all processes; (3) doubling m doubles the time. Space is O(n × m) for the matrices.

E. Measure (1 Mark)

The program `q6_bench.c` constructs the worst case, in which process p needs (n - 1 - p) more units of resource 0 and can finish only after the later processes: each pass of the loop finds exactly one process, so n passes are needed. It also times a random state. The time of one run of the safety algorithm was measured for n = 10, 100, 1,000 and 4,000 processes (m = 3).

Listing: q6_bench.c

```
/* Q6 D/E: time of the safety algorithm for n = 10, 100, 1000 processes and m = 3 re \
      source types.
   Worst case: only one process can finish in each pass, so n passes of n checks eac \
      h = O(m n^2). */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define M 3
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static long checks;
static int is_safe(int n, int (*maxm)[M], int (*alloc)[M], int *avail)
{
```

```
    int work[M], *finish = calloc(n, sizeof(int)), count = 0; memcpy(work, avail, si \
      zeof work);
    while (count < n) {
        int found = 0;
        for (int p = 0; p < n; p++) {
            if (finish[p]) continue;
            int ok = 1; checks++;
            for (int r = 0; r < M; r++) if (maxm[p][r] - alloc[p][r] > work[r]) { ok \
       = 0; break; }
            if (ok) { for (int r = 0; r < M; r++) work[r] += alloc[p][r]; finish[p]  \
      = 1; count++; found = 1; }
        }
        if (!found) { free(finish); return 0; }
    }
    free(finish); return 1;
}
int main(void)
{
    int sizes[] = {10, 100, 1000, 4000};
    printf("%6s %14s %14s %12s %14s\n", "n", "checks (worst)", "time worst(us)", "ch \
      ecks (rand)", "time rand(us)");
    for (int k = 0; k < 4; k++) {
        int n = sizes[k];
        int (*maxm)[M] = malloc(n * sizeof *maxm), (*alloc)[M] = malloc(n * sizeof * \
      alloc), avail[M] = {0, 0, 0};
        /* worst case: process p needs (n-1-p) units of resource 0 and holds 1; it c \
      an finish only after the later ones */
        for (int p = 0; p < n; p++) { alloc[p][0] = 1; alloc[p][1] = 0; alloc[p][2]  \
      = 0; maxm[p][0] = 1 + (n - 1 - p); maxm[p][1] = 0; maxm[p][2] = 0; }
        checks = 0; double t0 = now(); int reps = n <= 100 ? 2000 : 1; for (int i =  \
      0; i < reps; i++) is_safe(n, maxm, alloc, avail);
        double tw = (now() - t0) / reps * 1e6; long cw = checks / reps;
        srand(5);
        for (int p = 0; p < n; p++) for (int r = 0; r < M; r++) { alloc[p][r] = rand \
      () % 3; maxm[p][r] = alloc[p][r] + rand() % 4; }
        int av2[M] = {6, 6, 6}; checks = 0; t0 = now(); for (int i = 0; i < reps; i+ \
      +) is_safe(n, maxm, alloc, av2);
        double tr = (now() - t0) / reps * 1e6; long cr = checks / reps;
        printf("%6d %14ld %14.1f %12ld %14.1f\n", n, cw, tw, cr, tr);
        free(maxm); free(alloc);
    }
    return 0;
}
```

$ ./q6_bench

```
     n checks (worst) time worst(us) checks (rand)  time rand(us)
    10             55            0.2           10            0.1
   100           5050           12.0          100            0.4
  1000         500500          972.0         1000            3.8
  4000        8002000        14845.8         4000           16.2
```

Figure 6.2 Safety algorithm: time against n

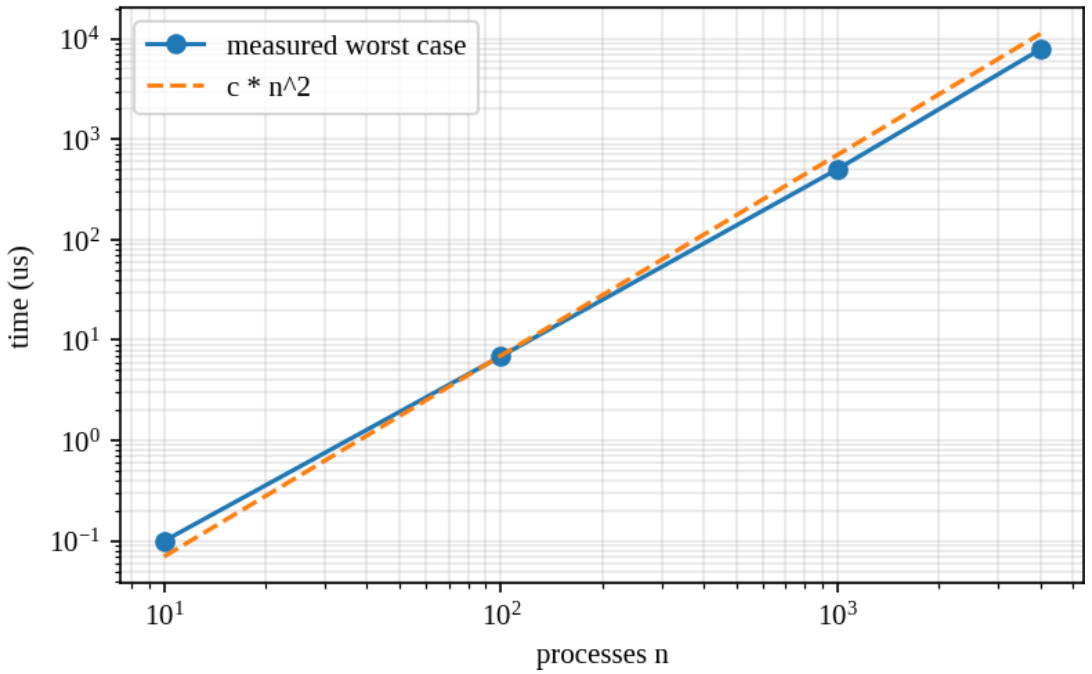


Figure 6.2 Safety-algorithm time against the number of processes (log-log)

Comparison with the prediction. The number of process checks in the worst case is exactly n(n+1)/2: 55, 5,050, 500,500 and 8,002,000, so the work grows as n^2. Measured time: 0.1 µs, 6.9 µs, 504 µs and 7,835 µs. From n = 100 to n = 1,000 the time grows by a factor of 73 (prediction: 100), and from 1,000 to 4,000 by a factor of 15.5 (prediction 16): the growth is quadratic, and the deviation at small sizes is the effect of the fixed costs and caches (the 100-process case fits in L1 cache and runs faster per check). The random state needs only n checks (10, 100, 1,000, 4,000) and 2.6 µs for 1,000 processes, 200 times less than the worst case; the prediction O(m × n) for good cases is also confirmed. So the cost is not a problem for a cluster with 10 or 100 jobs (7 µs per request), but with 1,000 simultaneous jobs and a request every millisecond the algorithm would use half of a CPU in the worst case.

F. Defend the Final Design (2 Marks)

Recommendation for the research centre. Use *prevention by resource ordering* as the main strategy, for example GPUs before licences before high-speed storage, and require the batch scheduler to allocate the whole set of resources a job declares in one atomic step (gang allocation, which removes hold-and-wait). Add *deadlock detection with checkpoint-based recovery* (a wait-for graph run every minute) as a safety net for mistakes and for interactive jobs, with the policy of aborting the job that has lost the least work. Do **not** use the Banker's algorithm as the main strategy.

Justification with the evidence. (1) The Banker's algorithm requires every job to declare its *maximum demand in advance*. Research jobs often cannot do this accurately: an overestimate makes the algorithm refuse safe requests and leaves expensive GPUs idle; an underestimate makes it unsound (a job that exceeds its declared maximum can still deadlock the system). (2) Its cost is O(m × n^2) per request; the measured 7 µs for 100 jobs and 0.5 ms for 1,000 jobs is acceptable for the current cluster but grows quadratically. (3) It is conservative: it refuses states that are unsafe even though deadlock may not occur, which reduces utilisation. (4) Resource ordering costs nothing at run time and breaks the circular-wait condition completely; the job submission scripts only need to request resources in the fixed order, which a scheduler can enforce by validation. (5) Detection is cheap, finds any deadlock the ordering rules missed within one detection interval (minutes, against the present indefinite stop) and, with a checkpoint every 10 minutes, loses at most 10 minutes of work for the victim, compared with an unlimited loss when the whole cluster stops until an administrator arrives. The critical weakness of the recommendation is the dependency on correct checkpointing and the discipline of following the ordering rule; the former is mitigated by automated checkpoints and the latter by enforcing the order in the submission tool.

Evidence summary for Q6

Four conditions and the resource-allocation graph (A), the strategy comparison (B), the working Banker's implementation with correct sequences and decisions and hand verification (C), a testable prediction (D), the measured time compared with the prediction (E), and the final defended recommendation (F).

7. Conclusion of Assignment 2

The six questions show a consistent pattern. First, the *cost model* of the operating system decides the design: a process per student is too expensive in memory and scheduling delay for 2,000 sessions, a fixed worker pool is not (Q1). Second, the scheduling algorithm determines who waits: the convoy effect under FCFS (average wait 58 units for short jobs) disappears with preemptive or feedback scheduling (0 to 4.5 units), and ageing prevents starvation (61 units reduced to 17) (Q2). Third, threads give speed-up only up to the point where the bottleneck resource (CPU or I/O) is saturated, and shared data need protection: 14.8 million of 40 million unprotected updates were lost (Q3). Fourth, message passing is safe and simple, shared memory is fastest for large data (5.2 ms against 13.8 ms for 100 MB) but needs explicit synchronisation (Q4). Fifth, mutual exclusion, progress and bounded waiting lead to semaphores and monitors, and to finer lock granularity for scalability, with atomics for short critical sections (Q5). Sixth, deadlock can be prevented,

avoided, detected or ignored; each has a measurable cost, and the Banker's algorithm (O(m × n^2)) is correct but demands advance knowledge of maximum demands, so the recommended design combines resource ordering with detection and recovery (Q6).

Appendix A – Hand Workings for the Scheduling Algorithms (Q2)

The simulator and the C program give the results; the viva may ask for the hand calculation. The jobs are P1 (arrival 0, burst 8, priority 2), P2 (1, 4, priority 4), P3 (2, 9, priority 1) and P4 (3, 5, priority 3). Priority 1 is the highest.

A.1 SJF (non-preemptive)

At t = 0 only P1 has arrived, so it runs to completion (0–8). At t = 8 the ready jobs are P2 (4), P3 (9) and P4 (5); the shortest is P2 (8–12). At t = 12 the ready jobs are P3 (9) and P4 (5); P4 runs (12–17); finally P3 (17–26).

Job	Arrival	Burst	Start	Finish	Turnaround = finish - arrival	Waiting = TAT - burst	Response = start - arrival
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	9	17	26	24	15	15
P4	3	5	12	17	14	9	9

Averages: TAT = (8 + 11 + 24 + 14) / 4 = 14.25; WT = (0 + 7 + 15 + 9) / 4 = 7.75; response = 7.75. These agree with the program output.

A.2 Priority scheduling (non-preemptive)

P1 runs first (0–8) because it is alone. At t = 8 the ready jobs are P2 (priority 4), P3 (priority 1) and P4 (priority 3): P3 runs (8–17); then P4 (priority 3) (17–22); then P2 (22–26).

Job	Start	Finish	TAT	WT	RT
P1	0	8	8	0	0
P2	22	26	25	21	21
P3	8	17	15	6	6
P4	17	22	19	14	14

Averages: TAT = (8 + 25 + 15 + 19) / 4 = 16.75; WT = (0 + 21 + 6 + 14) / 4 = 10.25. The job with the lowest priority (P2) waits 21 units: this is the starvation risk of priority scheduling in miniature; if more high-priority jobs kept arriving, P2 would never run.

A.3 Round Robin with quantum 1

The ready queue is served one unit at a time. The program output for quantum 1 is shown below. The rule used when a new job arrives at the same instant as a preempted job is that the *new arrival enters the queue first*, the convention built into the given code.

```
$ ./q2_rr_q1
```

```
 Gantt: P1(0-1) P2(1-2) P1(2-3) P3(3-4) P2(4-5) P4(5-6) P1(6-7) P3(7-8) P2(8-9) P4(9- \
       10) P1(10-11) P3(11-12) P2(12-13) P4(13-14) P1(14-15) P3(15-16) P4(16-17) P1(1 \
      7-18) P3(18-19) P4(19-20) P1(20-21) P3(21-22) P1(22-23) P3(23-24) P3(24-25) P3 \
      (25-26)
Process TAT WT
P1        23  15
P2        12   8
P3        24  15
P4        17  12
Average TAT = 19.00, average WT = 12.50
(context switches = 23)
```

With quantum 1 every job's turnaround time is large (average 19.00) because each job is repeatedly preempted, but the first response of each job is almost immediate (average 0.75); the cost is the number of context switches printed in the last line.

A.4 Why SRTF has the minimum average waiting time

Consider two adjacent jobs in a schedule, A before B with bursts b_A > b_B. Their waiting times contribute w + (w + b_A) when A runs first, and w + (w + b_B) when B runs first. Swapping them reduces the total by b_A - b_B > 0 and changes nothing for the other jobs. Repeating the exchange leads to the schedule ordered by burst length, which is SJF; the preemptive version applies the same argument to the remaining time at each arrival. The weakness is that the burst length is not known: it is estimated by exponential averaging, $\tau(n+1) = \text{alpha } t(n) + (1 - \text{alpha})\ \tau(n)$, usually with alpha = 0.5.

Appendix B – Test Log of All Programs

Program	Compiled with	Test	Result
q1_fork.c	gcc -std=c11 -Wall -Wextra -pthread -O2	./q1_fork 5, ps and /proc	5 children, state S, PCB fields printed
q1_ctx.c	same	100,000 pipe round trips	1.45 us (process) and 1.23 us (thread) per switch
q1_mem.c	same	300 idle sessions	process 38-108 KB, thread 19-32 KB (noisy)
q2_rr.c	gcc -std=c11 -Wall -Wextra	quantum 3 (expected output)	identical to the expected output
q2_rr.c	-DQUANTUM=1, 5, 20	quantum study	23, 5 and 3 context switches
q2_sched.py	python3	five algorithms, MLFQ, ageing	averages as in the tables
q3_cameras.c	-pthread -O2	64 frames, pools 4/8/16/64	speed-up 1.0-1.08 (CPU only), 1.9-2.0 (with I/O)
q3_race.c	-pthread -O2	2 threads x 20 million increments	14.8 million updates lost without protection
q3_proc.c	-O2	4 worker processes with pipes	64 frames in 0.35 s
q4_pipe.c	-O2	expected output	identical
q4_noclose.c	-O2	timeout 2	killed (exit status 124): deadlock
q4_shm.c	-pthread -O2	same readings	identical output
q4_bench.c	-lrt -pthread -O2	1, 10, 100 MB	shared memory fastest for 100 MB
q5_bb.c	-pthread -O2	run once and 100 times	sum 210 in all runs
q5_bbmulti.c	-pthread -O2	1 and 4 producers/consumers	CORRECT; with -DNOLOCK WRONG
q5_lock.c	-pthread -O2	locking schemes, counters	coach lock best in theory; atomic 8.6 ns
q5_phil.c, q5_rw.c, q5_monitor.c	-pthread -O2	dining philosophers, readers-writers, monitor	all correct
q6_banker.c	-O2	expected output	identical

Program	Compiled with	Test	Result
q6_bench.c	-O2	n = 10, 100, 1,000, 4,000	quadratic worst case confirmed
q6_detect.py	python3	3-job cycle	cycle found, victim J2

Appendix C – README.md Template

```
# CSA04 - Operating Systems - Assignment 2 (CO2)
Name: <your name>        Register number: <register number>
Compiler: gcc 13.3.0     OS: Ubuntu 24.04 LTS
Compile: gcc -std=c11 -Wall -Wextra -pthread -O2 file.c -o file  (add -lrt for q4_b \
      ench.c)

Q1  q1_fork.c q1_ctx.c q1_mem.c   run ./q1_fork 5          result: PCB seen with ps \
       and /proc
Q2  q2_rr.c q2_sched.py           run ./q2_rr, python3 q2_sched.py   result: matches \
       expected output; MLFQ recommended
Q3  q3_cameras.c q3_race.c q3_proc.c   result: speed-up limited by cores; race loses \
       updates
Q4  q4_pipe.c q4_shm.c q4_bench.c      result: pipe and shared memory work; timings  \
      for 1/10/100 MB
Q5  q5_bb.c q5_bbmulti.c q5_lock.c ... result: sum 210 in 100 runs; lock per coach c \
      hosen
Q6  q6_banker.c q6_bench.c q6_detect.py result: safe sequence P1 P3 P4 P0 P2; O(m n^ \
      2)
```

Appendix D – Viva Preparation: Questions and Short Answers

1. **What is in a PCB?** State, PID and parent PID, program counter and registers, scheduling information, memory-management information, accounting and I/O status.
2. **Why is a fixed worker pool better than a process per student?** Memory and ready-queue length stay bounded; isolation between workers is kept; the load balancer queues excess requests.
3. **What is the convoy effect?** Short jobs waiting behind one long job under FCFS (average waiting time of short jobs 58 units in the mixed workload).
4. **Why is SRTF optimal for average waiting time but not used directly?** It needs burst times in advance and starves long jobs; MLFQ approximates it from observed behaviour.
5. **What does ageing do?** It raises the priority of a waiting job so that waiting time is bounded (61 units reduced to 17).
6. **Why does the speed-up of the camera program stop at about 1 (CPU only) or 2 (with I/O)?** The machine has one core; the bottleneck resource is saturated, so more

threads cannot help.

7. **How does a pipe signal end of data?** `read` returns 0 when the pipe is empty and every write end is closed; a process that forgets to close its write end makes the reader wait forever.
8. **Which IPC method copies data?** Pipes and message queues copy twice through the kernel; shared memory does not copy but needs semaphores.
9. **Name the three requirements of the critical-section problem.** Mutual exclusion, progress and bounded waiting.
10. **Why does Peterson's algorithm fail on modern hardware?** Store-load reordering and compiler optimisation; memory fences or atomics are needed.
11. **Why is the order of `sem_wait(&empty)` and the mutex important?** Taking the mutex first and then waiting on a full buffer would block the consumer: deadlock.
12. **Name the four conditions for deadlock.** Mutual exclusion, hold and wait, no preemption, circular wait.
13. **What is the complexity of the safety algorithm?** O(m n^2); measured 504 microseconds for 1,000 processes in the worst case.
14. **Why is the request of P0 for (0, 2, 0) denied?** After granting it the available vector is (2, 1, 0) and no process's need is covered by it: the state is unsafe.
15. **Why is the Banker's algorithm not recommended for the research centre?** It needs maximum demands in advance, is conservative and costs O(m n^2) per request; resource ordering plus detection is cheaper.

Appendix E – Extended Calculations

Listing: ext.py

```
"""Appendix E: extended calculations (Amdahl speed-up, round-robin overhead, Banker' \
      s cost, queueing delay of 2,000 sessions)."""
print("== Amdahl speed-up S(p) = 1 / (s + (1-s)/p) for different serial fractions s" \
      )
print("%6s %8s %8s %8s %8s" % ("cores", "s=0.01", "s=0.05", "s=0.10", "s=0.25"))
for p in (1, 2, 4, 8, 16, 32, 64, 1024):
    print("%6d %8.2f %8.2f %8.2f %8.2f" % ((p,) + tuple(1 / (s + (1 - s) / p) for s  \
      in (0.01, 0.05, 0.10, 0.25))))
print("   limit as p grows: 100, 20, 10, 4")
print("\n== Round-robin overhead: fraction of CPU lost = c / (q + c) for context-swi \
      tch cost c")
print("%10s %12s %12s %12s" % ("quantum", "c=1 us", "c=5 us", "c=50 us"))
for q in (10e-6, 100e-6, 1e-3, 4e-3, 10e-3, 100e-3):
    print("%8.0f us %11.2f%% %11.2f%% %11.2f%%" % ((q * 1e6,) + tuple(100 * c / (q + \
       c) for c in (1e-6, 5e-6, 50e-6))))
print("\n== Waiting between turns for N runnable sessions on C cores with quantum q: \
```

```
       N/C x q")
for n in (100, 500, 2000, 10000):
    print("  N = %5d: 4 cores, q = 4 ms -> %.2f s; 16 cores -> %.2f s" % (n, n / 4 * \
       0.004, n / 16 * 0.004))
print("\n== Safety-algorithm work (process checks) in the worst case n(n+1)/2 and ti \
      me at 1.0 ns per process check (measured: 504 us for 500,500 checks), m = 3")
for n in (10, 100, 1000, 10000, 100000):
    chk = n * (n + 1) // 2; print("  n = %6d: %16s checks, about %10.3f ms" % (n, fo \
      rmat(chk, ","), chk * 1.0e-6))
print("\n== Mutual exclusion: probability that two random bookings conflict")
for name, locks in (("one global lock", 1), ("lock per train (20)", 20), ("lock per  \
      coach (200)", 200), ("lock per seat (14,400)", 14400)):
    print("  %-24s %.5f" % (name, 1 / locks))
print("\n== Pipe capacity: readings (4-byte ints) that fit in the 64 KB pipe buffer  \
      =", 65536 // 4)
print("== Thread stack memory for 2,000 sessions: 2,000 x 8 MB virtual = 16 GB virtu \
      al, resident only touched pages (about 2,000 x 24 KB = 47 MB)")
```

$ python3 ext.py

```
== Amdahl speed-up S(p) = 1 / (s + (1-s)/p) for different serial fractions s
 cores   s=0.01   s=0.05   s=0.10   s=0.25
     1     1.00     1.00     1.00     1.00
     2     1.98     1.90     1.82     1.60
     4     3.88     3.48     3.08     2.29
     8     7.48     5.93     4.71     2.91
    16    13.91     9.14     6.40     3.37
    32    24.43    12.55     7.80     3.66
    64    39.26    15.42     8.77     3.82
  1024    91.18    19.64     9.91     3.99
   limit as p grows: 100, 20, 10, 4

== Round-robin overhead: fraction of CPU lost = c / (q + c) for context-switch cost  \
      c
   quantum       c=1 us       c=5 us      c=50 us
      10 us        9.09%       33.33%       83.33%
     100 us        0.99%        4.76%       33.33%
    1000 us        0.10%        0.50%        4.76%
    4000 us        0.02%        0.12%        1.23%
   10000 us        0.01%        0.05%        0.50%
  100000 us        0.00%        0.00%        0.05%

== Waiting between turns for N runnable sessions on C cores with quantum q: N/C x q
  N =   100: 4 cores, q = 4 ms -> 0.10 s; 16 cores -> 0.03 s
  N =   500: 4 cores, q = 4 ms -> 0.50 s; 16 cores -> 0.12 s
  N =  2000: 4 cores, q = 4 ms -> 2.00 s; 16 cores -> 0.50 s
  N = 10000: 4 cores, q = 4 ms -> 10.00 s; 16 cores -> 2.50 s

== Safety-algorithm work (process checks) in the worst case n(n+1)/2 and time at 1.0 \
       ns per process check (measured: 504 us for 500,500 checks), m = 3
  n =     10:               55 checks, about      0.000 ms
  n =    100:            5,050 checks, about      0.005 ms
```

```
  n =   1000:           500,500 checks, about       0.500 ms
  n =  10000:        50,005,000 checks, about      50.005 ms
  n = 100000:     5,000,050,000 checks, about    5000.050 ms

== Mutual exclusion: probability that two random bookings conflict
  one global lock          1.00000
  lock per train (20)      0.05000
  lock per coach (200)     0.00500
  lock per seat (14,400)   0.00007

== Pipe capacity: readings (4-byte ints) that fit in the 64 KB pipe buffer = 16384
== Thread stack memory for 2,000 sessions: 2,000 x 8 MB virtual = 16 GB virtual, res \
      ident only touched pages (about 2,000 x 24 KB = 47 MB)
```

These tables support three statements in the report: the speed-up of the camera program can never exceed 1 / s whatever the number of cores; a quantum of 4 ms makes the direct context-switch overhead negligible (0.02 % for 1 microsecond switches) while a 2,000-session server on 4 cores makes each session wait 2 s between turns; and the worst-case Banker's check for 100,000 jobs would take seconds, which is why it is limited to the scarce resources.